

CS2006: 計算機組織

**Computer Abstractions and
Technology**



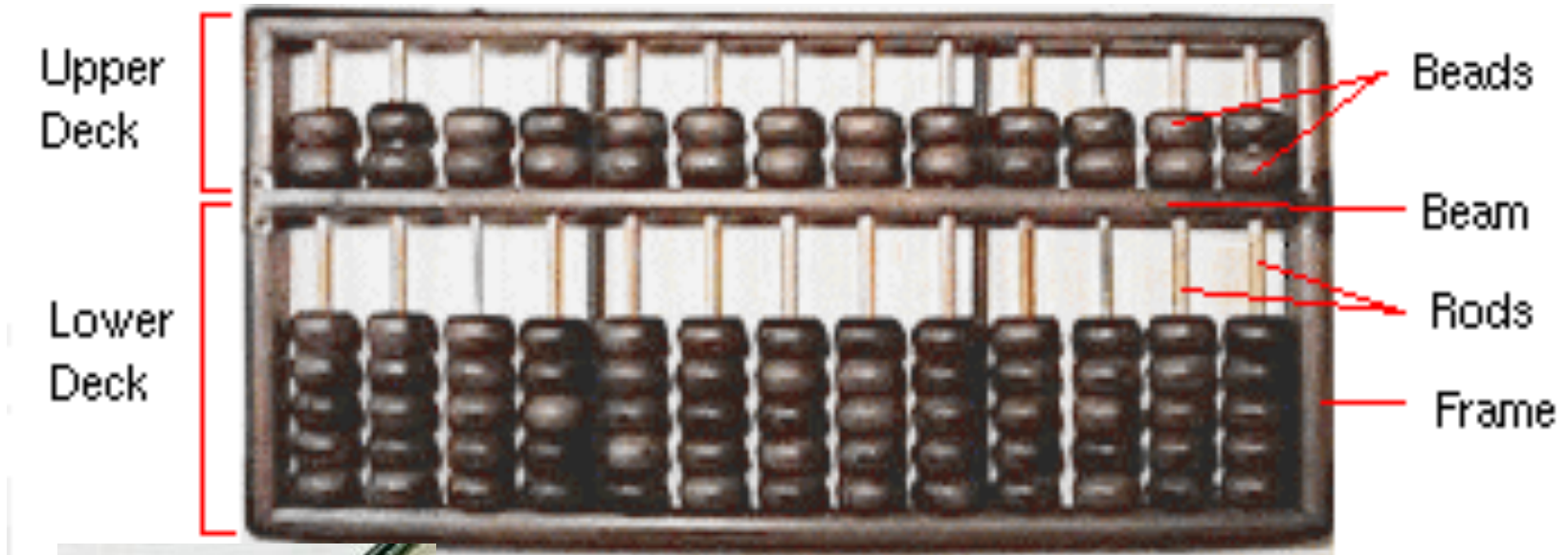
Outline



- ◆ **Computer: A historical perspective**
- ◆ **Moore's Law**
- ◆ **Abstractions**
- ◆ **Technology**
 - **Performance**
 - **Definition**
 - **CPU performance**
 - **Amdahl's Law**
 - **Benchmarking**
 - **Power wall**
 - **A trend from uniprocessors to multiprocessors**
 - **Manufacturing cost**

電腦是什麼時候發展出來的？

大約一千三百多年前...



為什麼我們不稱它為「電腦」？

電動算盤

那麼「電腦」到底是什麼？

- ◆ A device that computes, especially a **programmable** electronic machine that performs high-speed mathematical or logical operations or that assembles, stores, correlates, or otherwise processes information
-- *The American Heritage Dictionary of the English Language*, 4th Edition, 2000

其實歷史上已有許多計算裝置發展出來

- ◆ Special-purpose versus general-purpose
- ◆ Non-programmable versus programmable
- ◆ Scientific versus office data processing
- ◆ Mechanical, electromechanical, electronic, ...

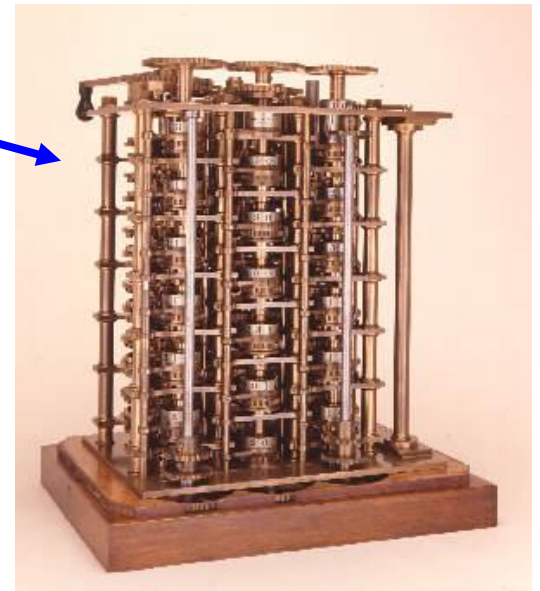
人口普查 海軍火炮射擊 多項式計算



Tabulating machine
(H. Hollerith, 1889)



Harvard Mark I
(IBM, H. Aiken, 1944)
Computer Abstractions and Technology-6



Difference Engine
(C. Babbage, 1822)

Computer Organization

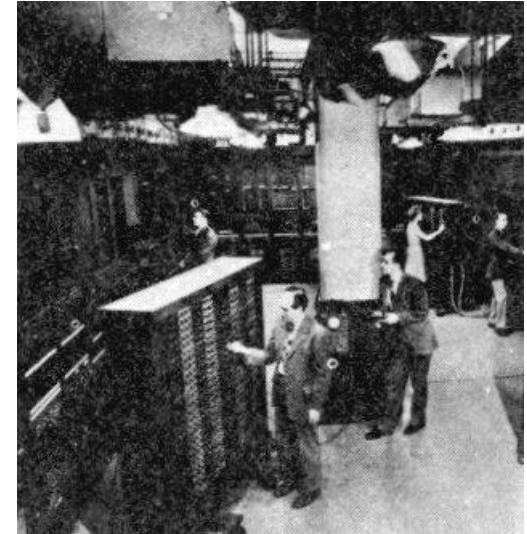
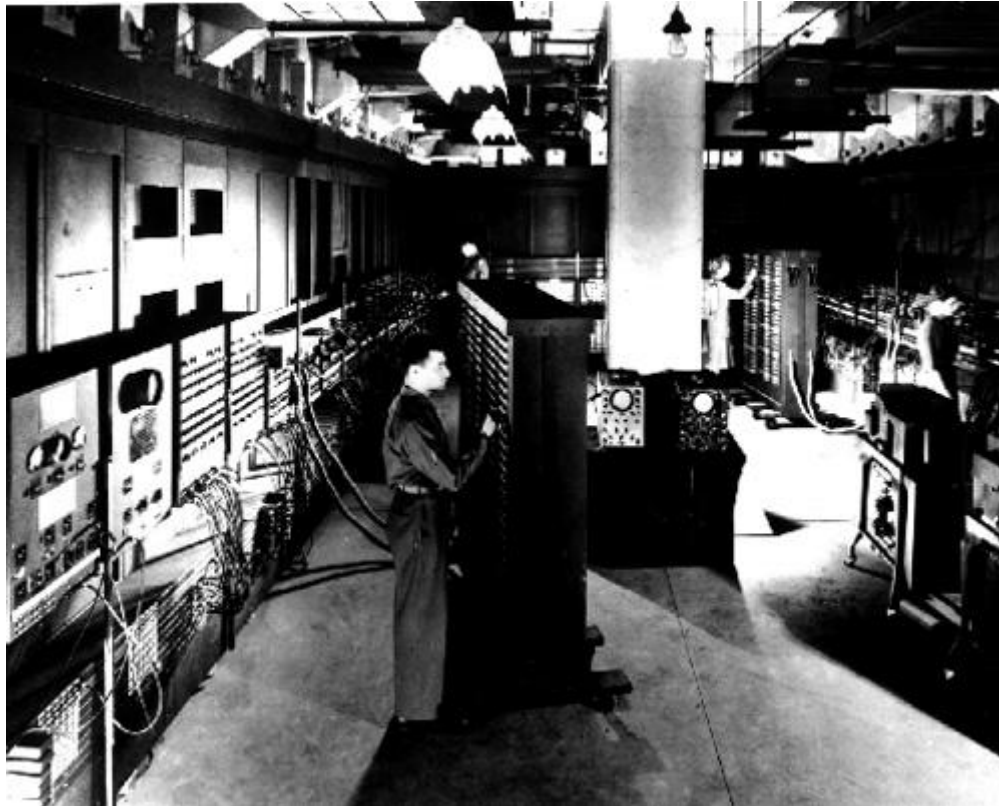
第一部
全電子式 (electronic)
可程式 (programmable)
一般用途 (general purpose)
的電腦
是什麼時候發展出來的？

第一部「電」腦

- ◆ 一般認為：**ENIAC (*Electronic Numerical Integrator and Calculator*)**
- ◆ **Work started in 1943 in Moore School of Electrical Engineering at the University of Pennsylvania, by John Mauchly and J. Presper Eckert**
- ◆ **Completed in 1946**
- ◆ **約25公尺長、2.5公尺高**
- ◆ **20 10-digit registers, each 2 feet**
- ◆ **使用18,000個真空管 (electronic switches, 1906年發明)** →
- ◆ **每秒執行1900個加法**
- ◆ **Programming manually by plugging cables and setting switches**

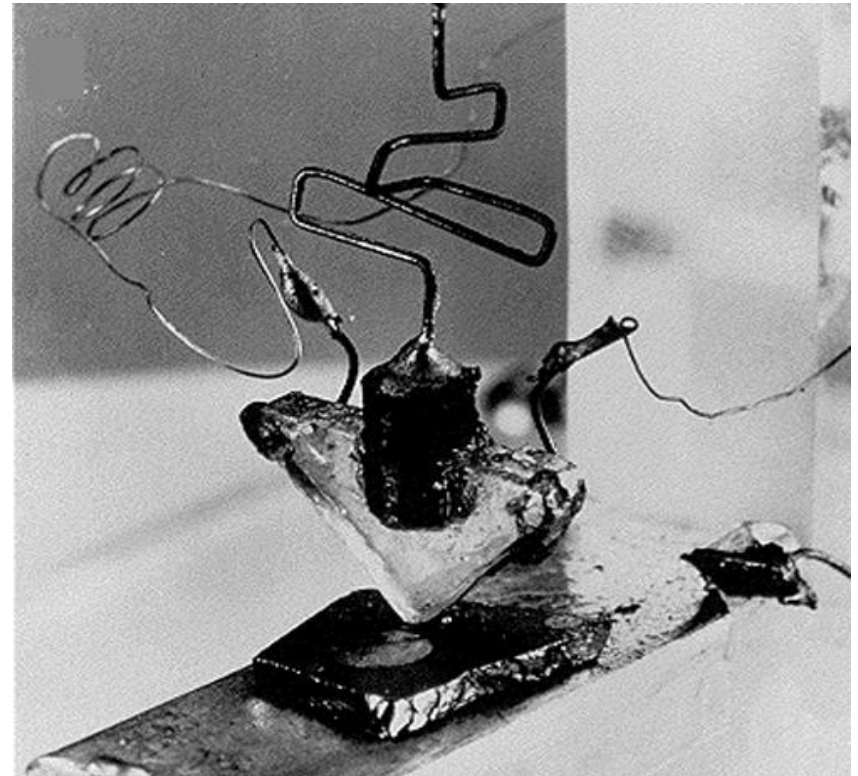
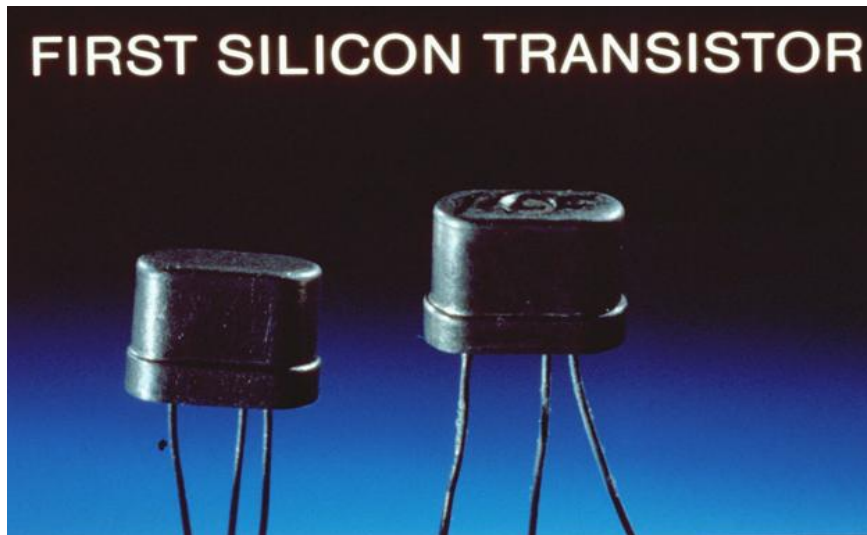


ENIAC



大約同一時期，人們發明了電晶體

- ◆ **By W. Shockley, J. Bardeen, W. Brattain of Bell Lab. in 1947**
 - **Much more reliable than vacuum tubes**
 - **Electronic switches in “solids”**



不久後電腦開始商品化



UNIVAC (Remington-Rand, 1951)

主要用途為商務、辦公室自動化
其次為科學計算



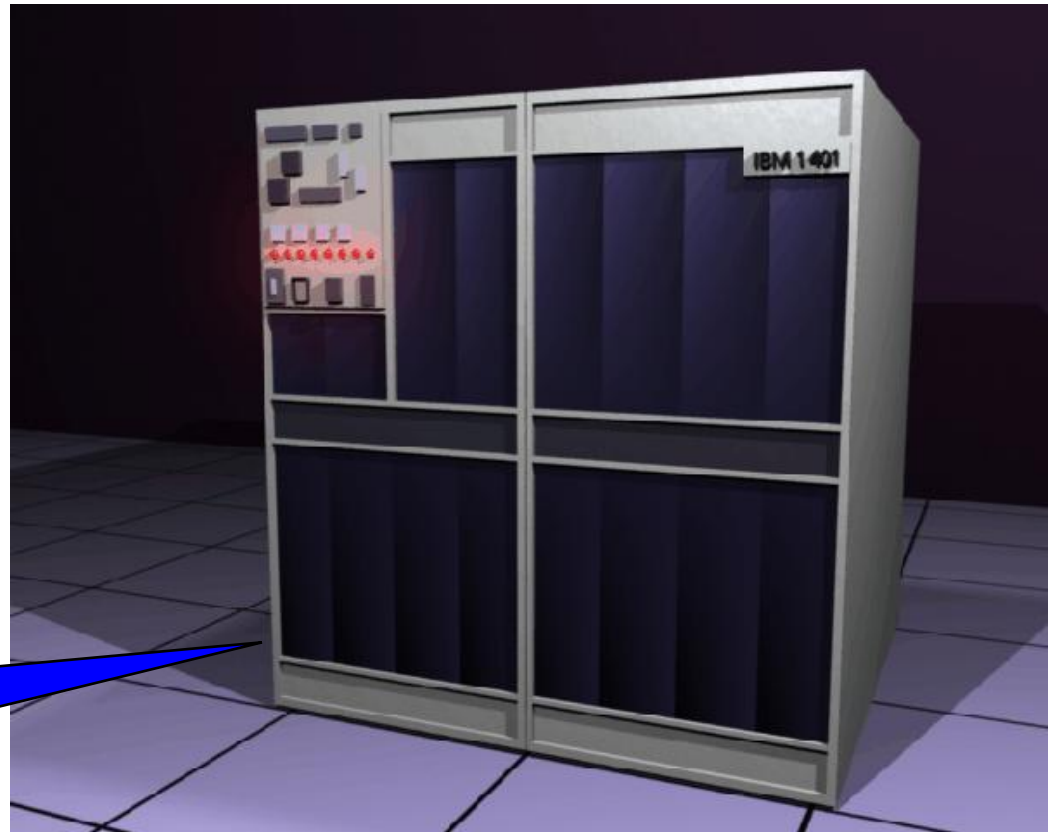
IBM 701 (IBM, 1952)

使用電晶體的電腦也跟著出現

◆ Ex.: IBM 1401 (IBM, 1959)

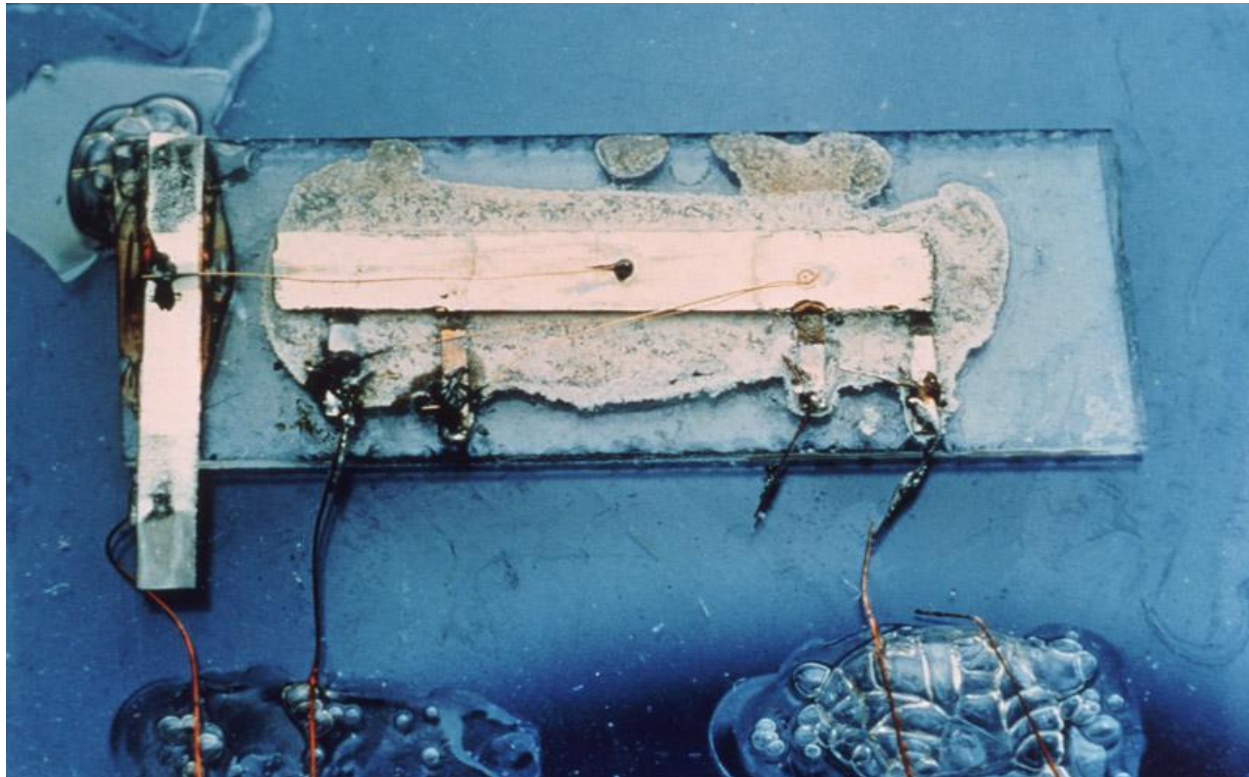


This is how
IBM is called
"Big Blue"!



電腦元件的另一大突破是IC

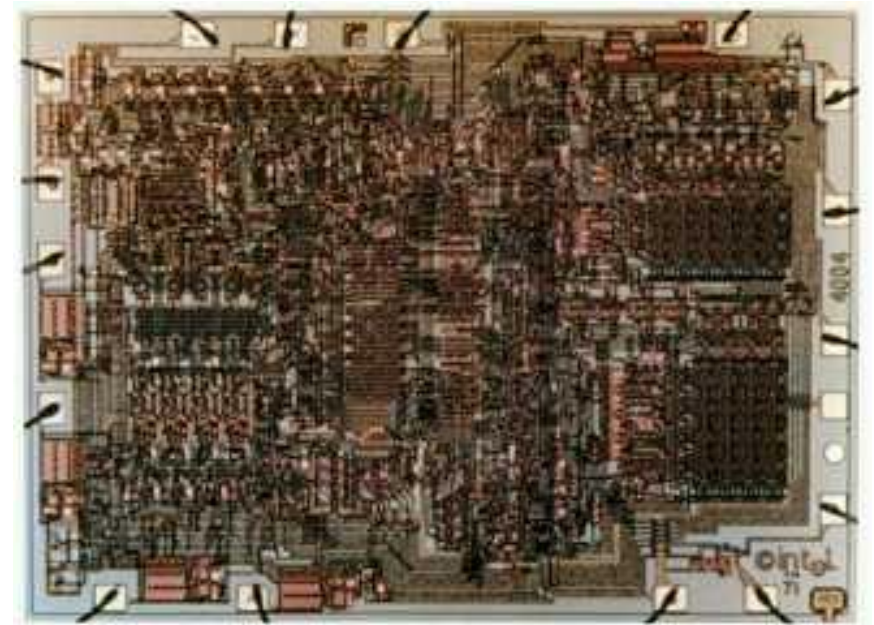
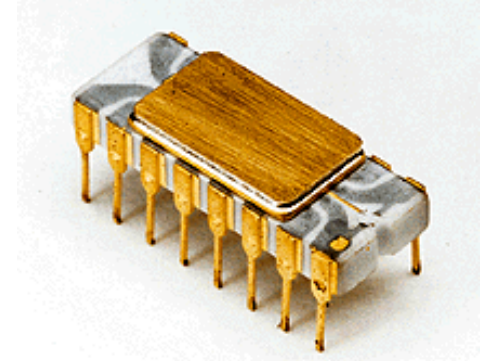
- ◆ 1958年德州儀器公司的**Jack Kilby**: integrated a transistor with resistors and capacitors on a single semiconductor chip, which is a monolithic IC



當更多的電晶體能放入IC後...

◆ 1971年第一個微處理器：Intel 4004

- 108 KHz, 0.06 MIPS
- 2300 transistors (10 microns)
- Bus width: 4 bits
- Memory addr.: 640 bytes
- For Busicom calculator (original commission was 12 chips → Intel MCS-4)
 - 4001 ROM
 - 4002 RAM
 - 4003 shift register
 - 4004 microprocessor



微處理器造就了...

- ◆ **1977年Apple II: Steve Jobs, Steve Wozniak**
Motorola 6502 CPU, 48Kb RAM



以及PC

- ◆ 1981年IBM PC: Intel 8088, 4.77MHz, 16Kb RAM, two 160Kb floppy disks

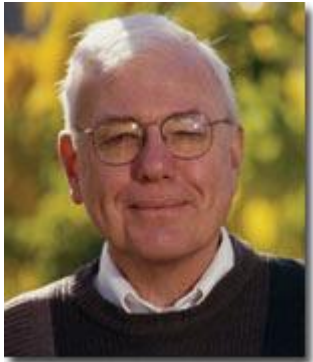


Microsoft Corporation, 1978

也造就了微軟

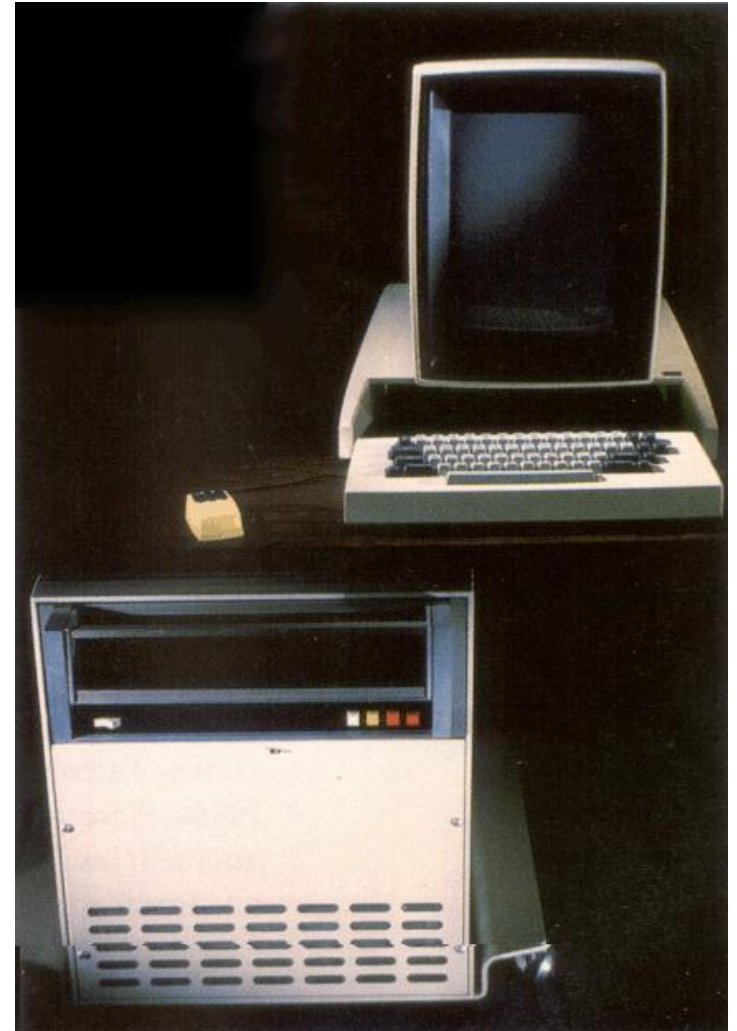
一些週邊設備也早已發展出來

- ◆ **1973: Researchers at Xerox PARC developed an experimental PC: **Alto****
 - Mouse, Ethernet, bit-mapped graphics, icons, menus, WYSIWYG editing
- ◆ **Hosted the invention of:**
 - Local-area networking
 - Laser printing
 - All of modern client / server distributed computing



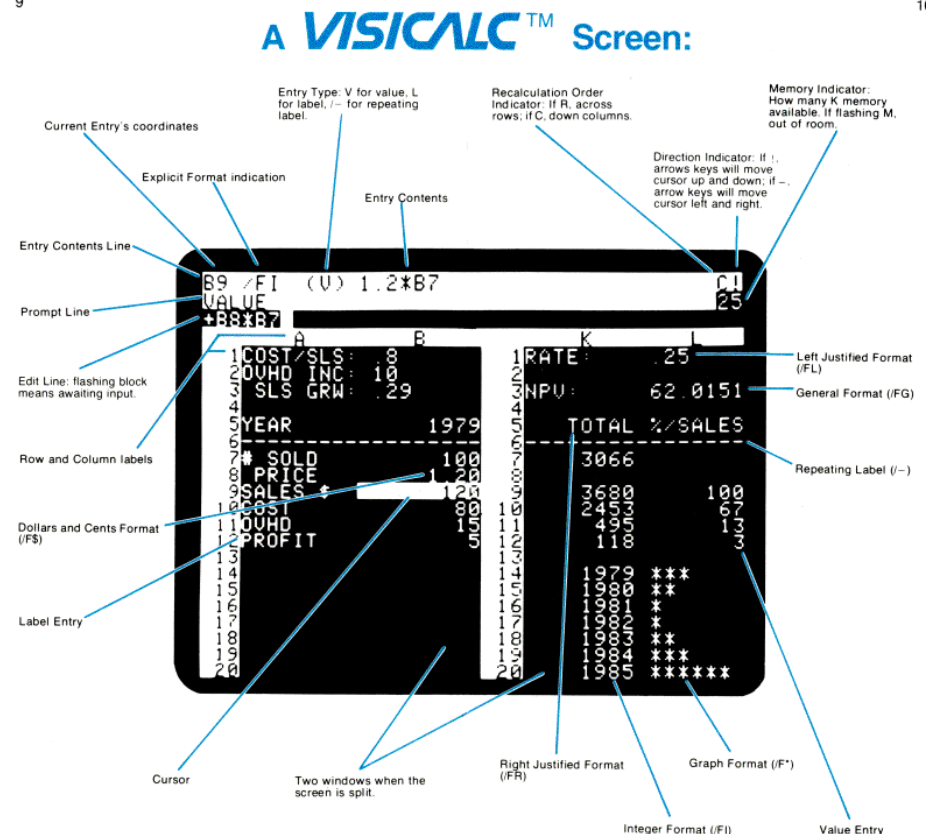
Charles P. Thacker

2009 Turing Award
\$250,000 USD



讓PC成為真正有用的東西 - 應用程式

- ◆ 1979: 1st electronic spreadsheet (VisiCalc for Apple II) by Don Bricklin and Bob Franston
 - “The killer app for early PCs”
 - Followed by dBASE II, ...



人們也先後發展出許多其他東西...



80年代，IC的集成進入VLSI

- ◆ **New processor architecture was introduced: RISC (*Reduced Instruction Set Computer*)**
 - IBM: John Cocke
 - UC Berkeley: David Patterson
 - Stanford: John Hennessy
- ◆ **Commercial RISC processors around 1985**
 - MIPS: MIPS
 - ARM: ARM
 - Sun: Sparc
 - IBM: Power RISC
 - HP: PA-RISC
 - DEC: Alpha
- ◆ **They compete with CISC (complex instruction set computer) processors, mainly Intel x86 processors**



後來的故事 ...

後**PC**的時代已經來臨
軟體與硬體協同設計成為新戰場
客製化計算機結構方興未艾



The Computer Revolution

- ◆ **Progress in computer technology**
 - **Underpinned by Moore's Law**
- ◆ **Makes novel applications feasible**
 - **Cell phones**
 - **Human genome project**
 - **Cloud computing**
 - **Artificial intelligence**
 - **Autonomous cars**
 - **Cyber physical system**
- ◆ **Computers are pervasive**

Classes of Computers

- ◆ **Desktop computers**
 - **General purpose, variety of software**
 - **Subject to cost/performance tradeoff**
- ◆ **Server computers**
 - **Network based**
 - **High capacity, performance, reliability**
 - **Range from small servers to building sized**
- ◆ **Embedded computers**
 - **Hidden as components of systems**
 - **Stringent power/performance/cost constraints**

Computer Progress Supported/Driven by Market and Usage

- ◆ **Applications drive machine “balance”**
 - Numerical simulations: floating-point, memory BW
 - Transaction processing: I/O, integer performance
 - Media processing: low-precision ‘pixel’ arithmetic
- ◆ **Applications drive machine performance**
 - What if my computer runs all my software very fast?
 - Programs use increasing amount of memory:
 - 1.5X-2X per year, or 0.5-1 addressing bit per year
 - High-level programming languages replace assembly languages → compilers are important
 - Compiler and architecture work together
- ◆ **Effects of compatibility and ease of use**
- ◆ **Effects of market demands and market share**
 - Can investment in R&D, production be paid off?

Computer Usage: General Purpose (PC and Server)

- ◆ **Uses: commercial (int.), scientific (FP, graphics), home (integer, audio, video, graphics)**
 - **Software compatibility is the most important factor**
 - **Short product life; higher price and profit margin**
 - **OS issue: OS serves another interface above arch.**
 - **Effects of OS developments on architecture**
 - **RISC-based Unix workstation vs. x86-based PC:**
 - Units sold is only 1% of PC's
 - Emphasize more on performance than on price
 - Survive only if performance is high enough?
 - Effects of Linux-based PCs?
- ◆ **Future:**
 - **Use increased transistors for performance, human interface (multimedia), bandwidth, monitoring**

Computer Usage: Embedded

- ♦ A computer inside another device used for running one predetermined application

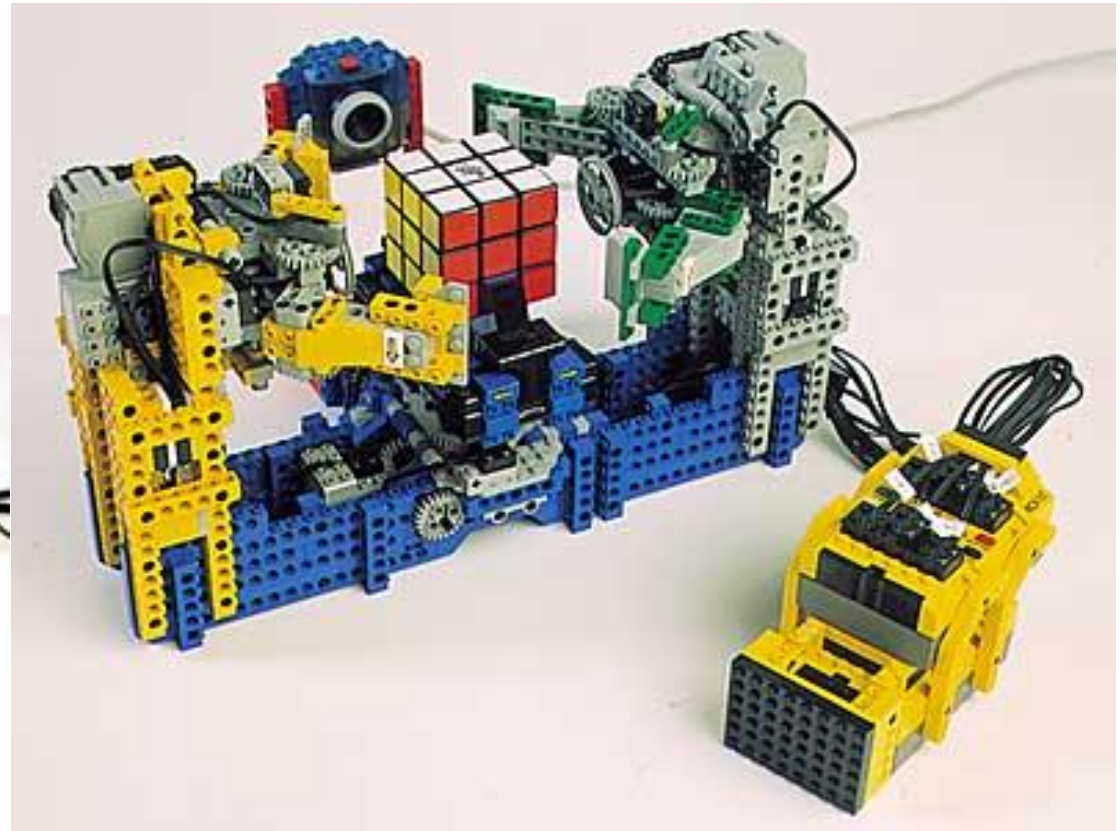
(e.g., printer, disk); consumer (e.g., game, CD player, PDA)

Lego Mindstorms

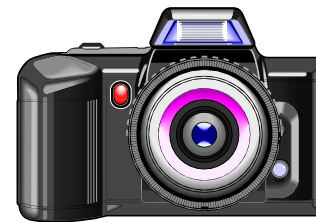
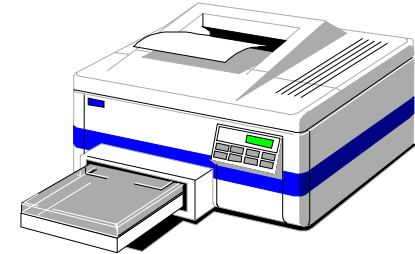
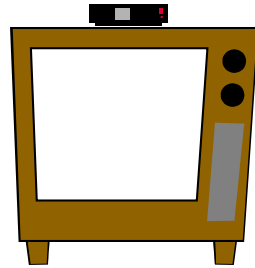


Robotic command explorer:
A “Programmable Brick”,
Hitachi H8 CPU (8-bit), 32KB RAM,
LCD, batteries,
infrared transmitter/receiver,
4 control buttons, 6 connectors

它可以做什麼？



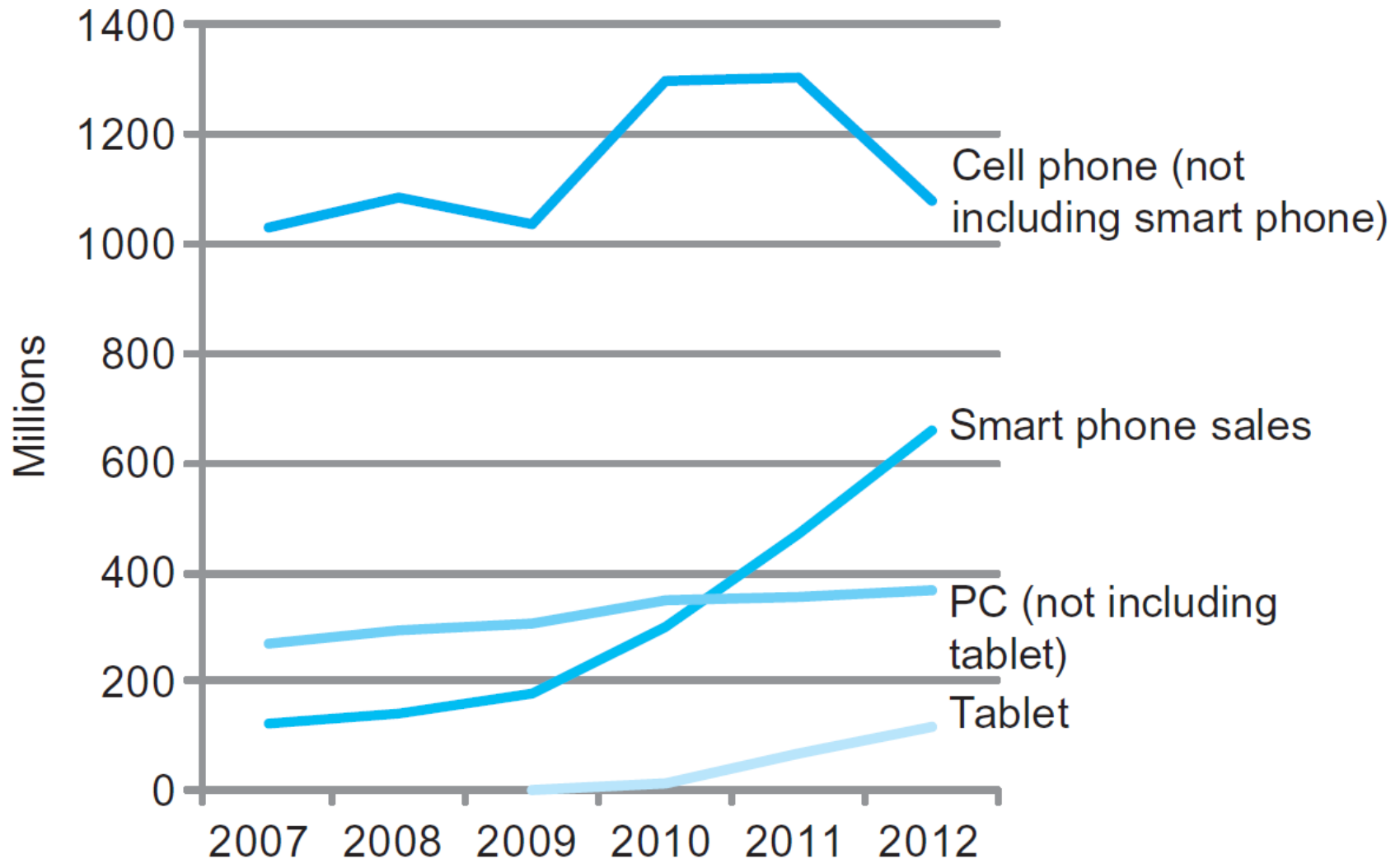
生活裡的應用比比皆是



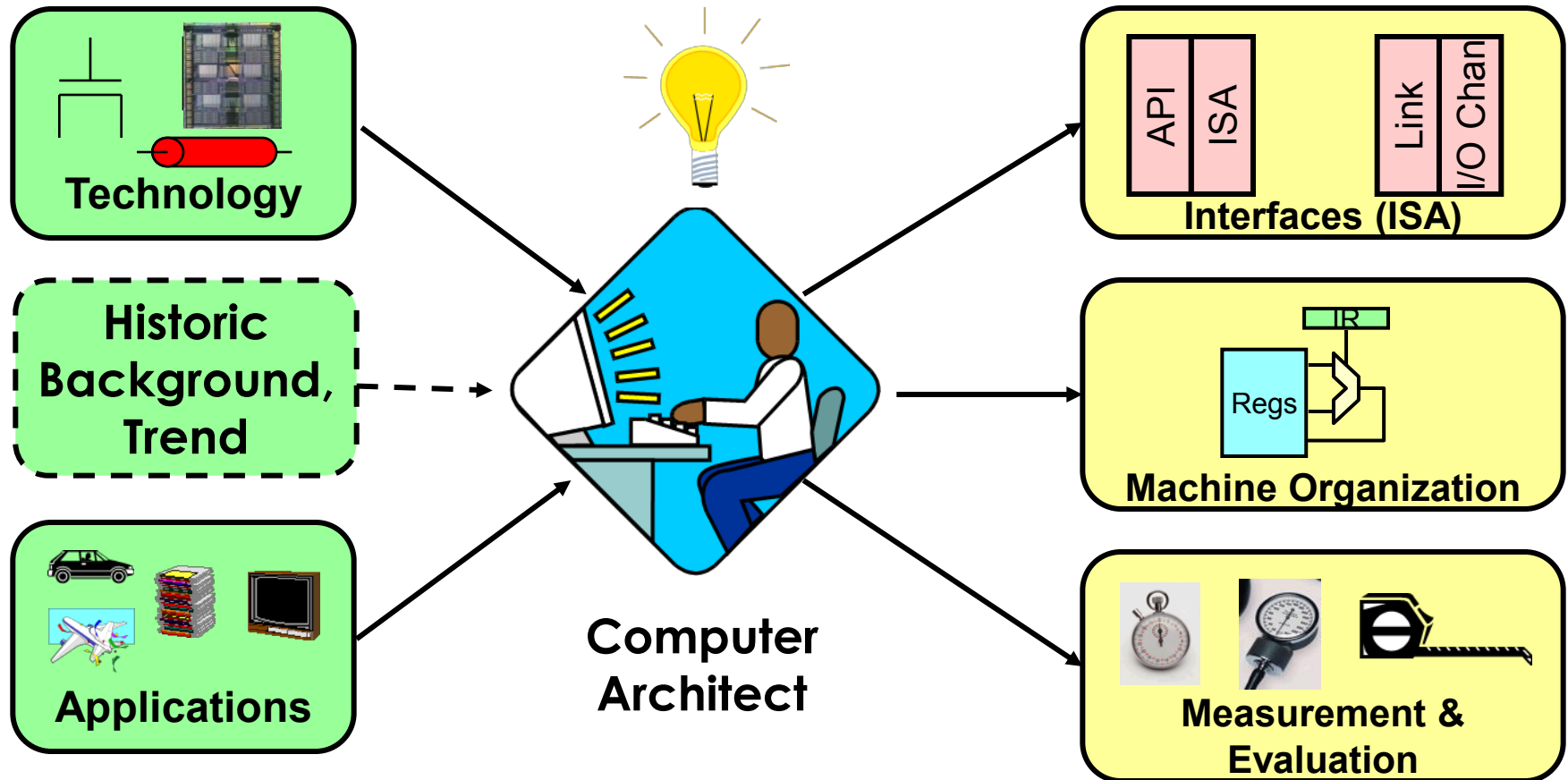
Embedded Computers

- ◆ **Typically w/o FP or MMU, but integrating various peripheral functions, ex: DSP**
 - Large variety in ISA, performance, on-chip peripherals
 - Compatibility is non-issue, new ISA easy to enter, low power become important
- ◆ **More architectures and survive longer: 4- or 8-bit microprocessor still in use (8-bit for cost-sensitive, 32-bit for performance)**
- ◆ **Large volume sale (billions) at low price (\$5-\$40)**
- ◆ **Use of microprocessor:**
 - 1995 #1: x86; #2: 6800; #3: Hitachi SuperH (Sega)
 - 2002 #1: ARM #2: x86; #3: Motorola 6800
- ◆ **Trend: lower cost, more functionality**
 - **System-on-chip, μ P core on ASIC**

The Processor Market

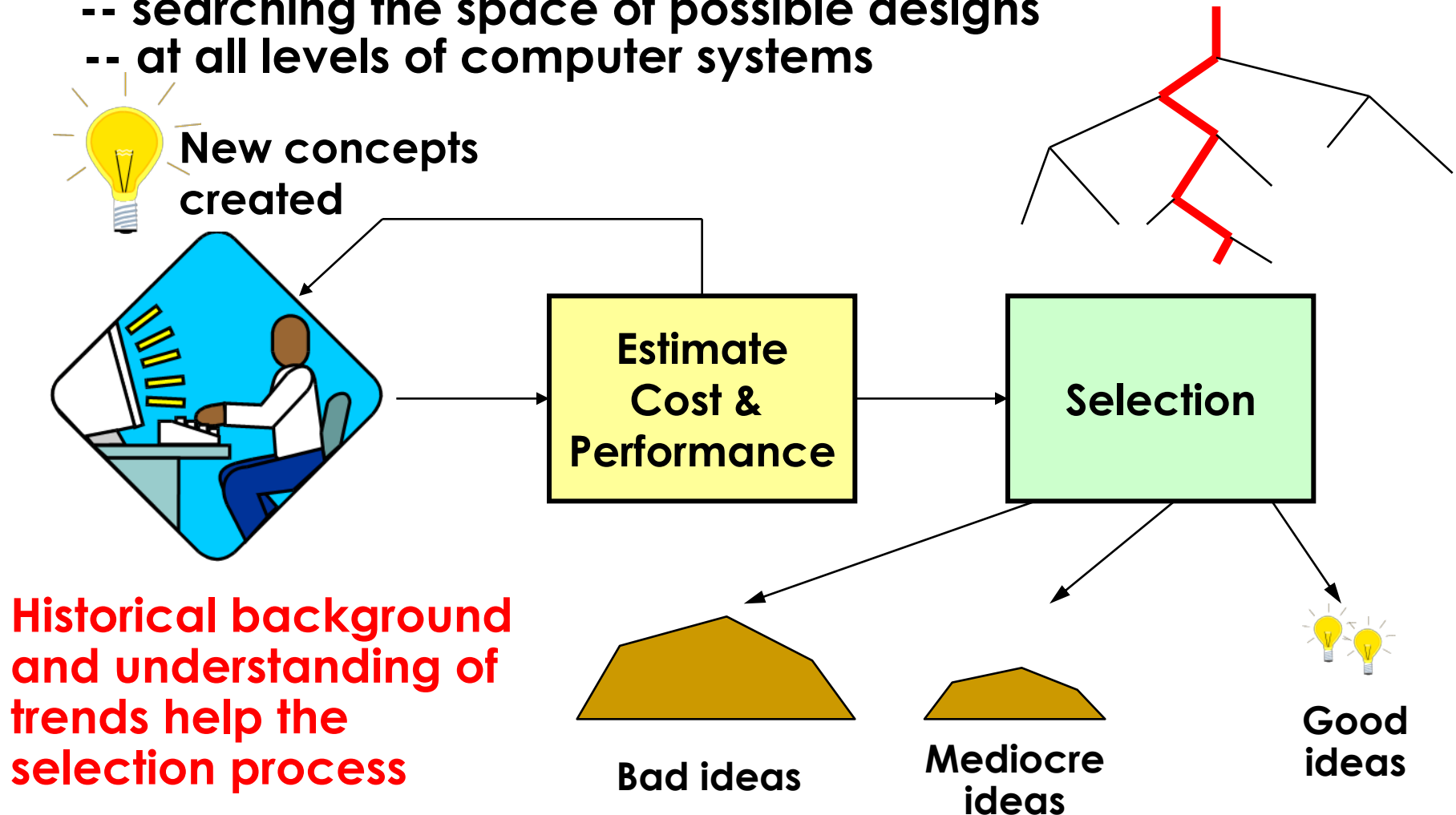


Why Do I Need to Know History?



In Fact, Architecture Design Is an Iterative Process

- searching the space of possible designs
- at all levels of computer systems



What You Will Learn

- ◆ **How programs are translated into the machine language**
 - **And how the hardware executes them**
- ◆ **The hardware/software interface**
- ◆ **What determines program performance**
 - **And how it can be improved**
- ◆ **How hardware designers improve performance**
- ◆ **What is parallel processing**

Understanding Performance

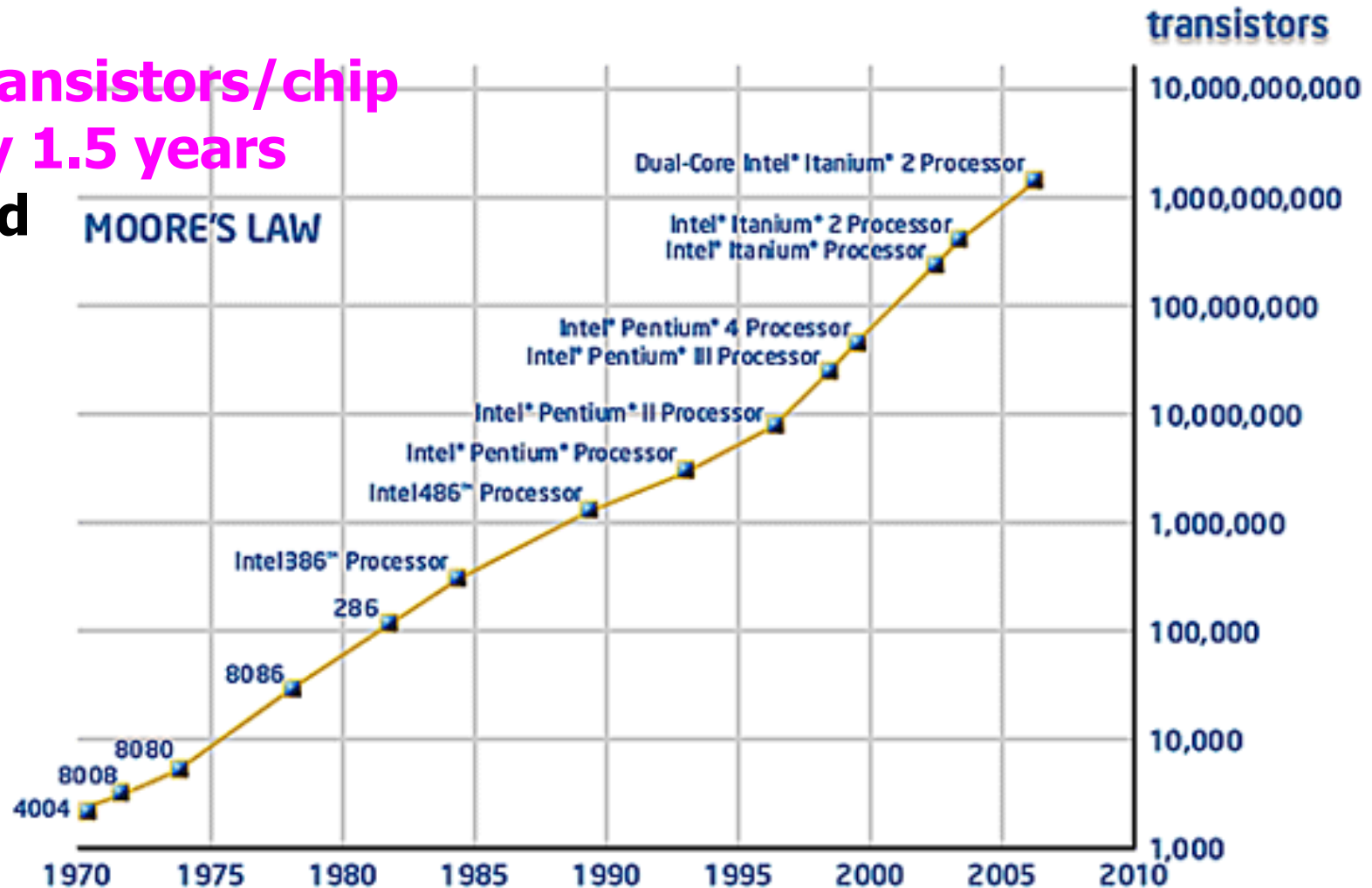
- ◆ **Algorithm**
 - **Determines number of operations executed**
- ◆ **Programming language, compiler, architecture**
 - **Determine number of machine instructions executed per operation**
- ◆ **Processor and memory system**
 - **Determine how fast instructions are executed**
- ◆ **I/O system (including OS)**
 - **Determines how fast I/O operations are executed**

Outline

- ◆ **Computer: A historical perspective**
- ◆ **Moore's Law**
- ◆ **Abstractions**
- ◆ **Technology**
 - **Performance**
 - **Definition**
 - **CPU performance**
 - **Amdahl's Law**
 - **Benchmarking**
 - **Power wall**
 - **A trend from uniprocessors to multiprocessors**
 - **Manufacturing cost**

Technology Trends: Microprocessor Capacity

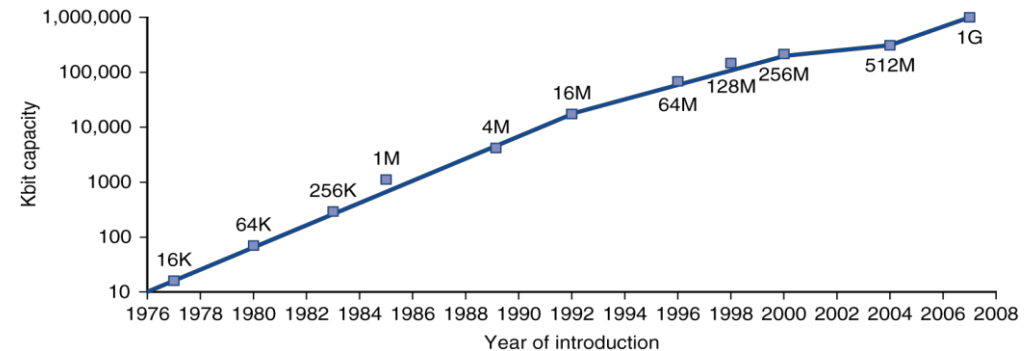
2X transistors/chip
every 1.5 years
Called



Technology Trends

◆ Electronics technology continues to evolve

- Increased capacity and performance
- Reduced cost



DRAM capacity

Year	Technology	Relative performance/cost
1951	Vacuum tube	1
1965	Transistor	35
1975	Integrated circuit (IC)	900
1995	Very large scale IC (VLSI)	2,400,000
2005	Ultra large scale IC	6,200,000,000

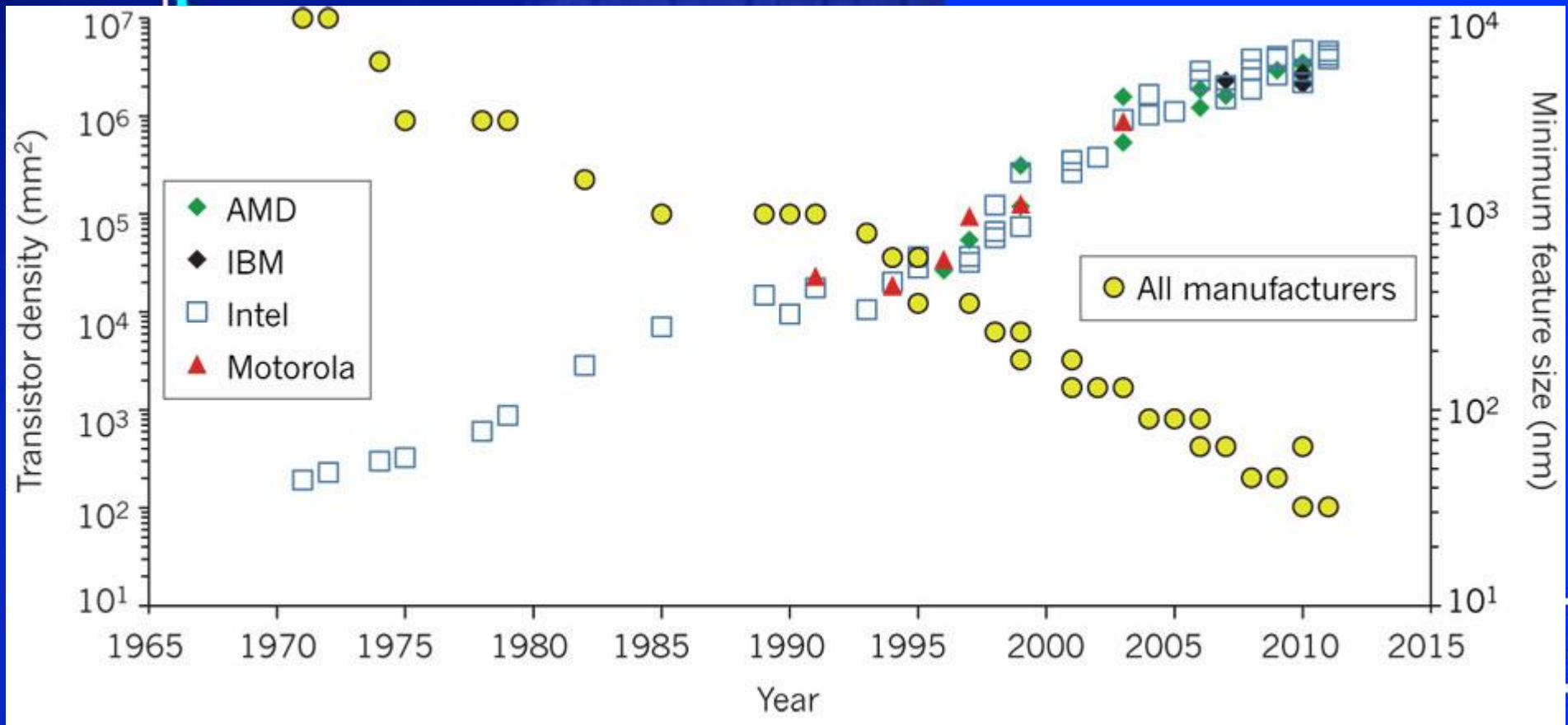
Minimum Feature Size

Feature Size
(microns)

100 **

Examples

Human hair, 100 μm



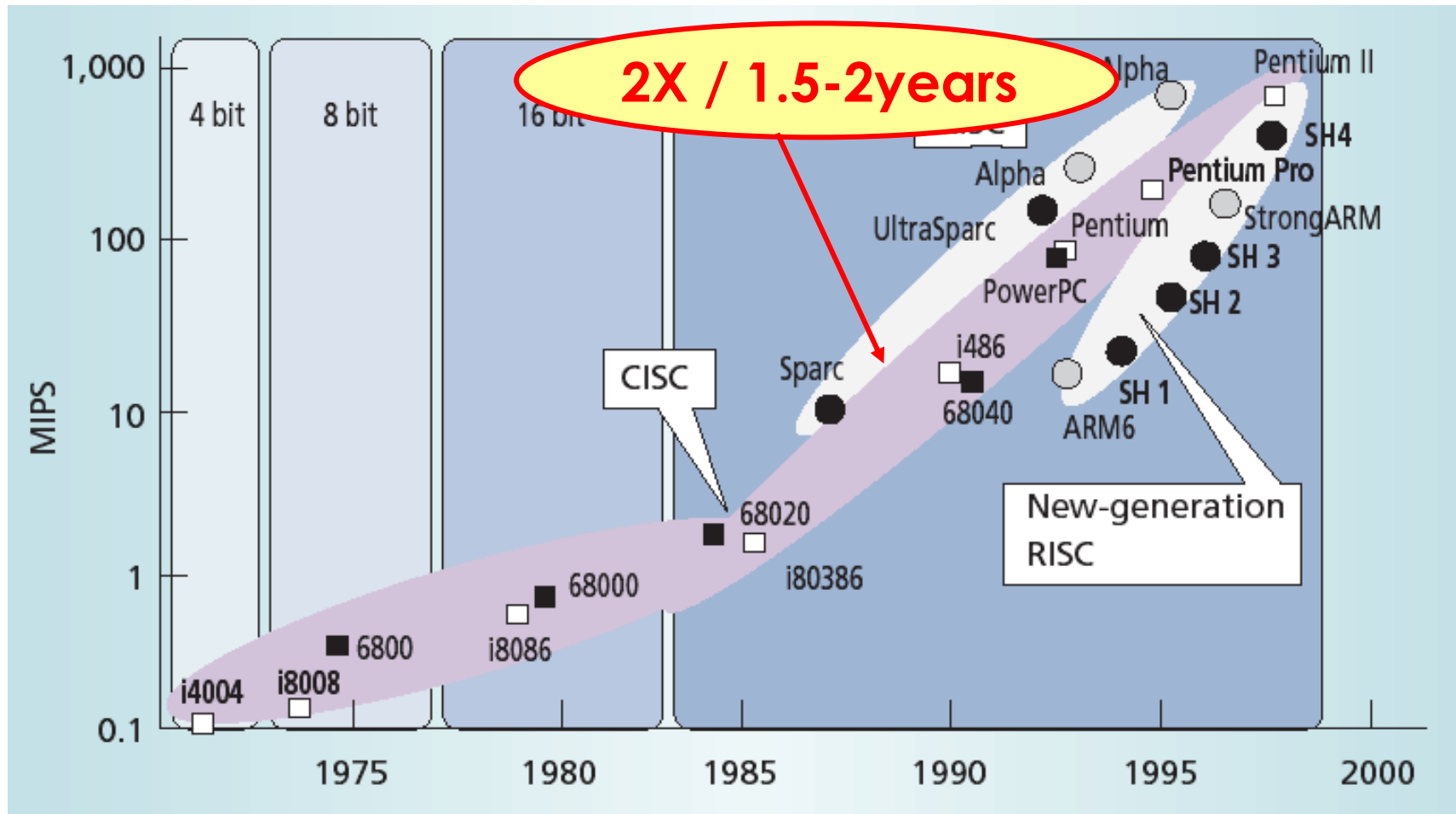
Buckyball, 0.001 μm

** Planar Transistor; remaining data points are ICs.

Source: Intel, post '96 trend data provided by SIA
International Technology Roadmap for Semiconductors (ITRS)

^ [ITRS DRAM Half-Pitch vs. Intel "Lithography"]

Let's Start with Processor Performance



("The Cooler the Better: New Directions in the Nomadic Ages," *Computer*, April 2001.)

Why Such Changes?

Several factors:

- ♦ **VLSI technology:**
clock rate, power, transistors per chip
 enable
- ♦ **Computer architecture:**
pipeline, cache, MMX, instructions per cycle
 supported by
- ♦ **Mass market:**
market share, revenue, applications

Technology → Dramatic Change

◆ Processor

- 2X in speed every 1.5 years; 100X in last decade

◆ Memory

- DRAM capacity: 4x / 3 years; 64X size in last decade
- Cost per bit: improves about 25% per year

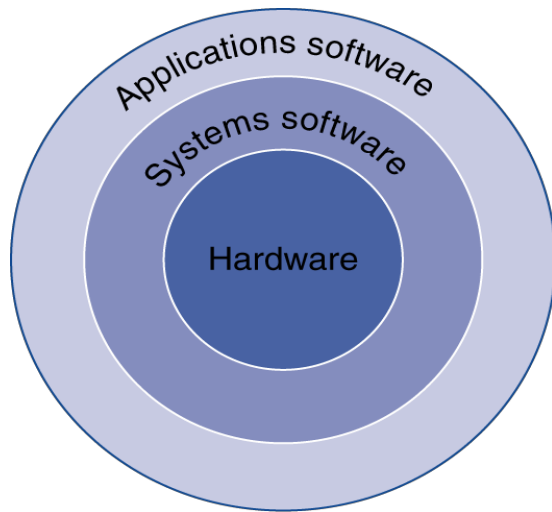
◆ Disk

- Capacity: 1.5~2X every year; 120X in last decade
- Cost per bit: improves about 100% per year

Outline

- ◆ **Computer: A historical perspective**
- ◆ **Moore's Law**
- ◆ **Abstractions**
- ◆ **Technology**
 - **Performance**
 - **Definition**
 - **CPU performance**
 - **Amdahl's Law**
 - **Benchmarking**
 - **Power wall**
 - **A trend from uniprocessors to multiprocessors**
 - **Manufacturing cost**

Below Your Program



- ◆ **Application software**
 - **Written in high-level language (HLL)**
- ◆ **System software**
 - **Compiler: translates HLL code to machine code**
 - **Operating System: service code**
 - **Handling input/output**
 - **Managing memory and storage**
 - **Scheduling tasks & sharing resources**
- ◆ **Hardware**
 - **Processor, memory, I/O controllers**

Levels of Program Code

- ◆ **High-level language**
 - Level of abstraction closer to problem domain
 - Provides for productivity and portability
- ◆ **Assembly language**
 - Textual representation of instructions
- ◆ **Hardware representation**
 - Binary digits (bits)
 - Encoded instructions and data

High-level
language
program
(in C)

```
swap(int v[], int k)
{int temp;
  temp = v[k];
  v[k] = v[k+1];
  v[k+1] = temp;
}
```

Compiler

Assembly
language
program
(for MIPS)

```
swap:
  muli $2, $5, 4
  add  $2, $4, $2
  lw   $15, 0($2)
  lw   $16, 4($2)
  sw   $16, 0($2)
  sw   $15, 4($2)
  jr   $31
```

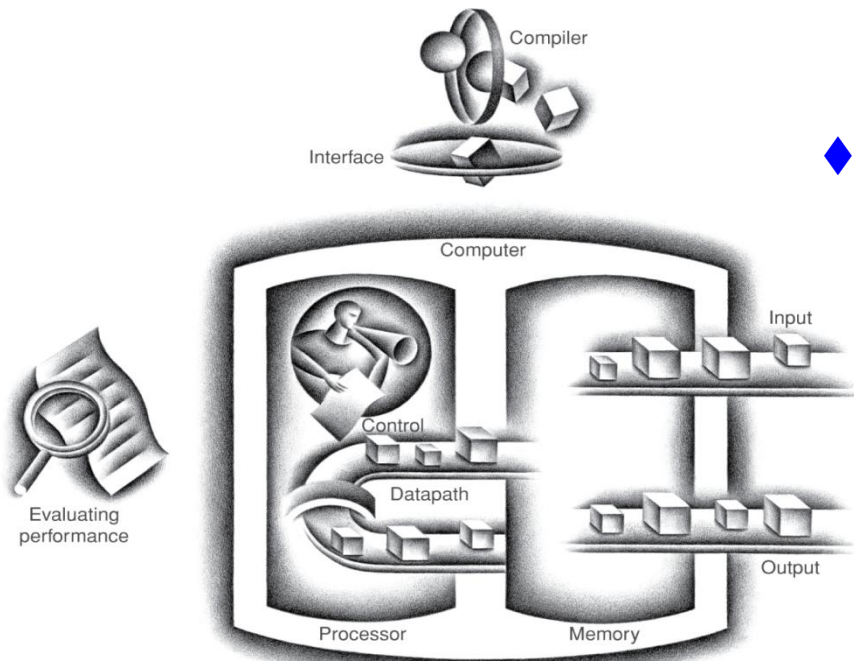
Assembler

Binary machine
language
program
(for MIPS)

```
000000001010000100000000000011000
000000000000110000001100000100001
100011000110001000000000000000000
100011001111001000000000000000100
101011001111001000000000000000000
101011000110001000000000000000100
00000011111000000000000000001000
```

Components of a Computer

The BIG Picture



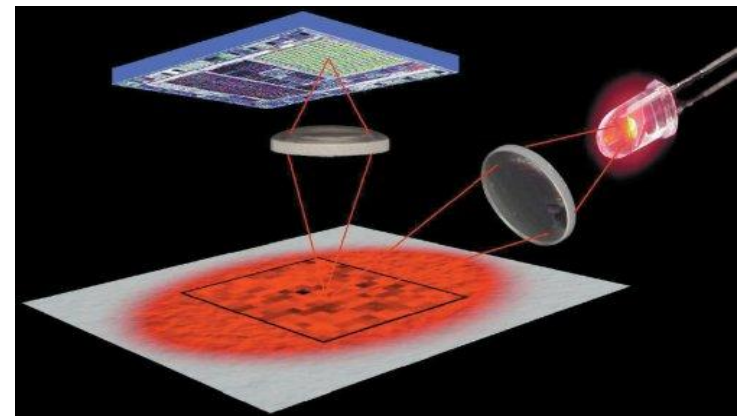
- ◆ Same components for all kinds of computer
 - Desktop, server, embedded
- ◆ Input/output includes
 - User-interface devices
 - Display, keyboard, mouse
 - Storage devices
 - Hard disk, CD/DVD, flash
 - Network adapters
 - For communicating with other computers

Anatomy of a Computer



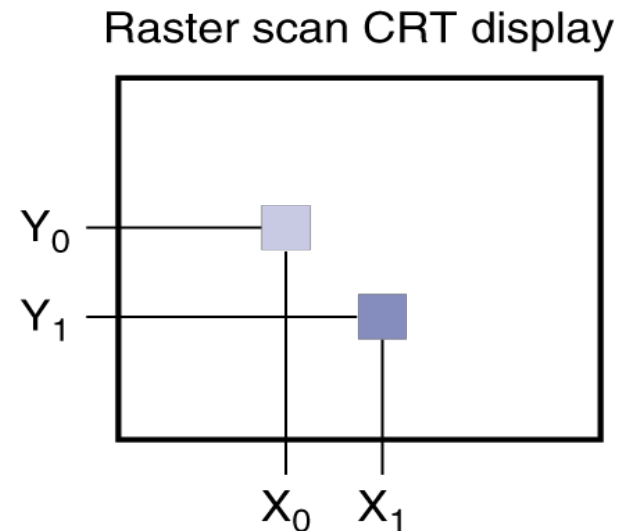
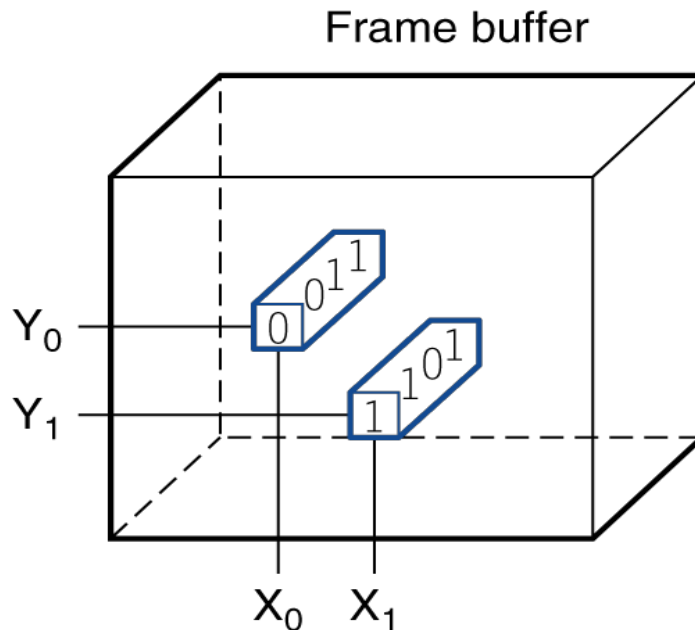
Anatomy of a Mouse

- ◆ **Optical mouse**
 - **LED illuminates desktop**
 - **Small low-res camera**
 - **Basic image processor**
 - Looks for x, y movement
 - **Buttons & wheel**
- ◆ **Supersedes roller-ball mechanical mouse**

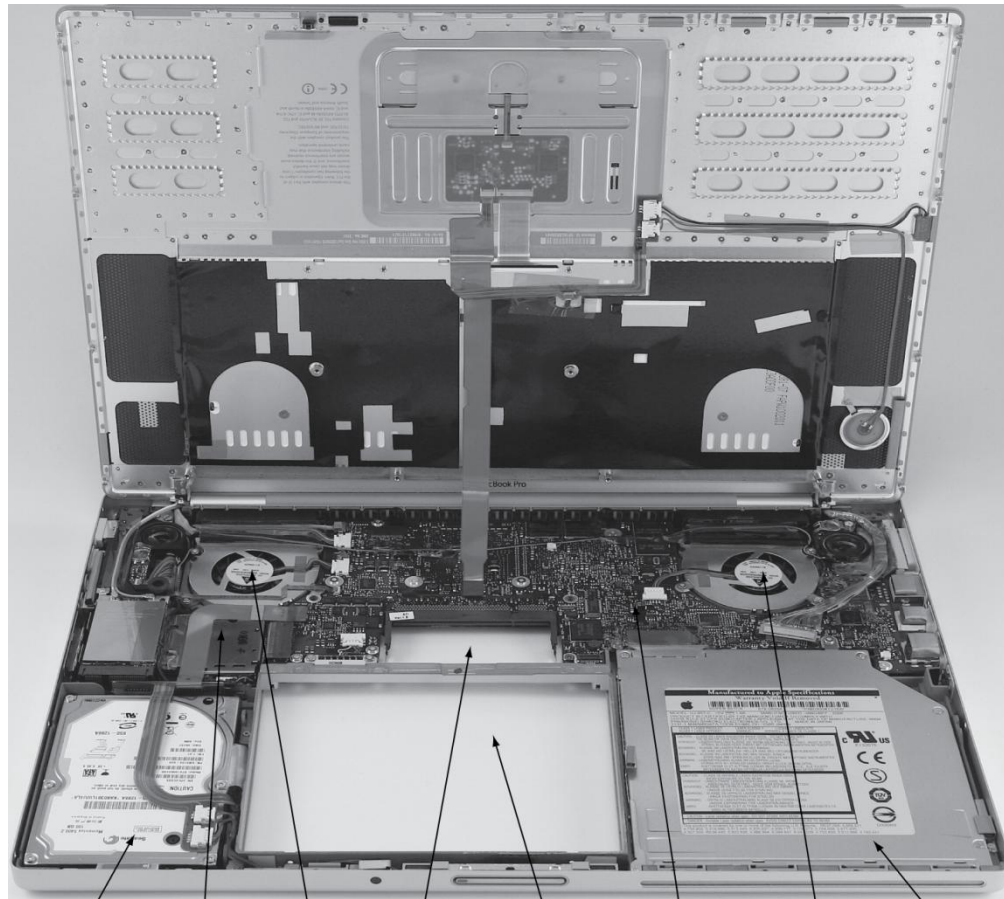


Through the Looking Glass

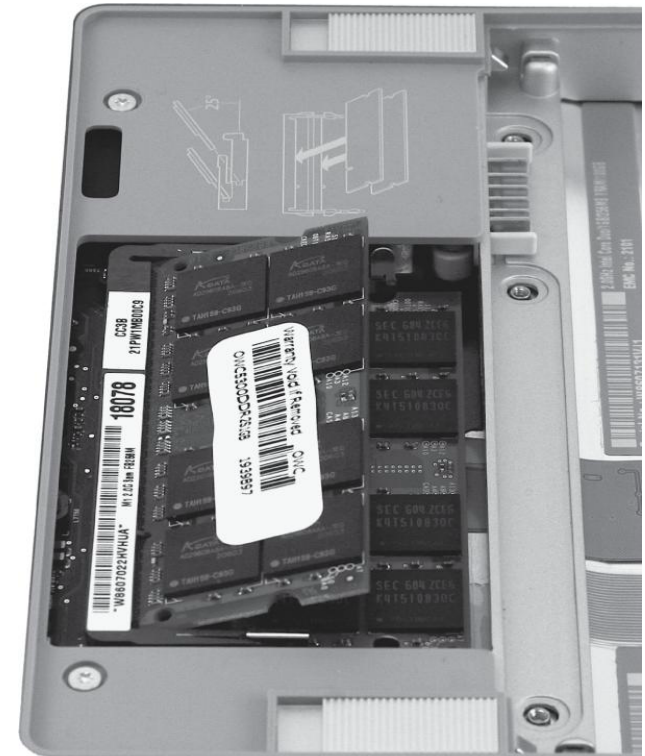
- ◆ **LCD screen: picture elements (pixels)**
 - **Mirrors content of frame buffer memory**



Opening the Box



Hard drive Processor Fan with cover Spot for memory DIMMs Spot for battery Motherboard Fan with cover DVD drive

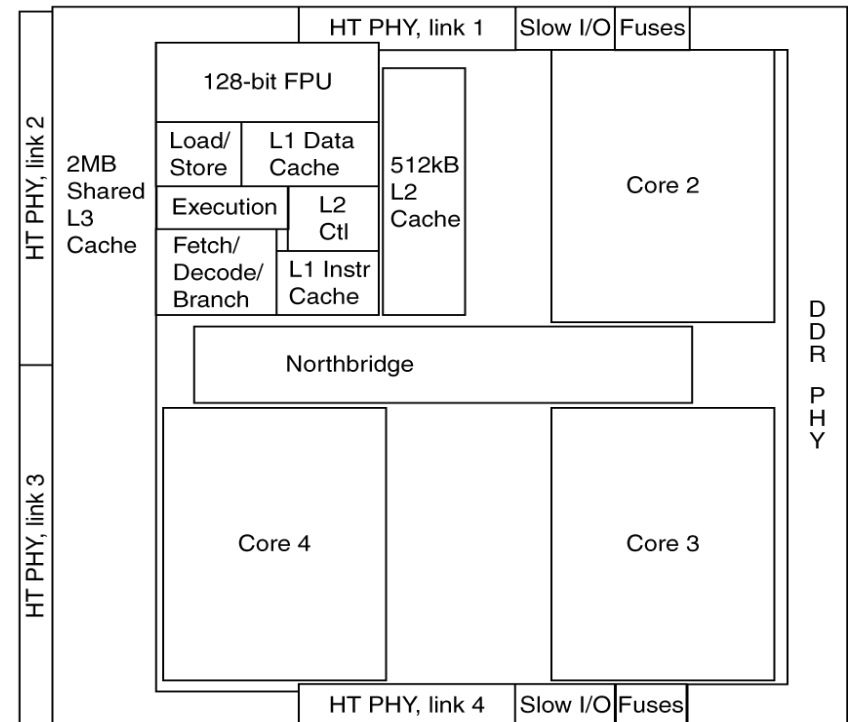
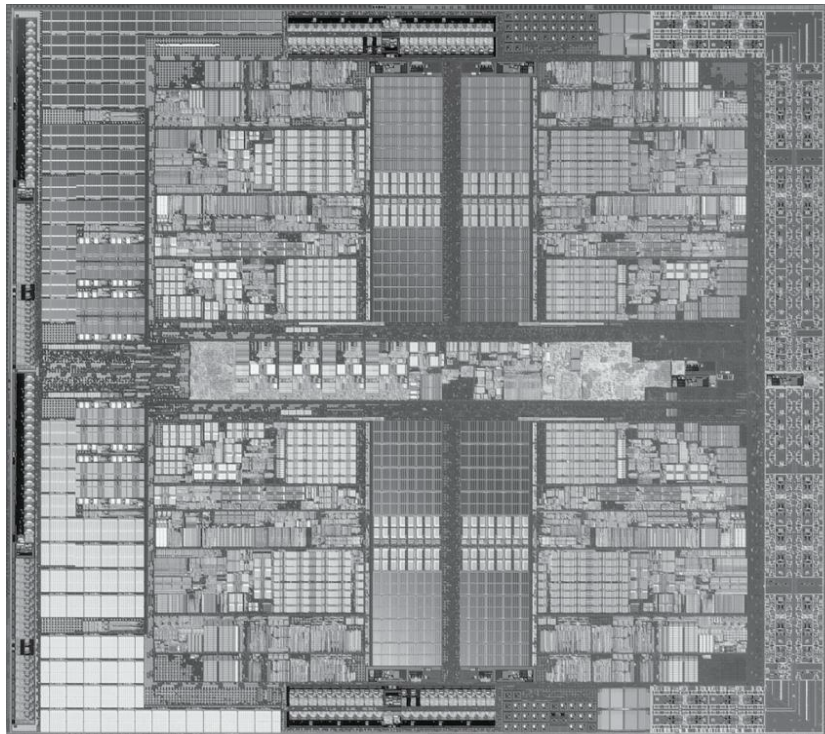


Inside the Processor (CPU)

- ◆ **Datapath: performs operations on data**
- ◆ **Control: sequences datapath, memory, ...**
- ◆ **Cache memory**
 - **Small fast SRAM memory for immediate access to data**

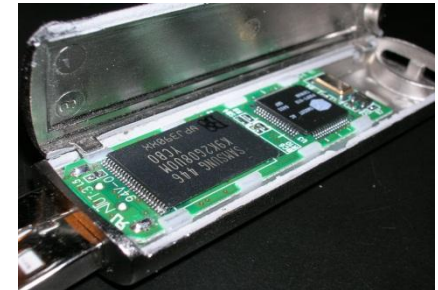
Inside the Processor

◆ AMD Barcelona: 4 processor cores



A Safe Place for Data

- ◆ **Volatile main memory**
 - Loses instructions and data when power off
- ◆ **Non-volatile secondary memory**
 - Magnetic disk
 - Flash memory
 - Optical disk (CDROM, DVD)



Networks

- ◆ **Communication and resource sharing**
- ◆ **Local area network (LAN): Ethernet**
 - **Within a building**
- ◆ **Wide area network (WAN): the Internet**
- ◆ **Wireless network: WiFi, Bluetooth**



Outline

- ◆ **Computer: A historical perspective**
- ◆ **Moore's Law**
- ◆ **Abstractions**
- ◆ **Technology**
 - **Performance**
 - **Definition**
 - **CPU performance**
 - **Amdahl's Law**
 - **Benchmarking**
 - **Power wall**
 - **A trend from uniprocessors to multiprocessors**
 - **Manufacturing cost**

哪一架飛機的效能比較好？



Concorde:

- Capacity: 132 persons
- Range: 4000 miles
- Cruising speed: 1320 mph (Mach 2.02) at 60,000 ft

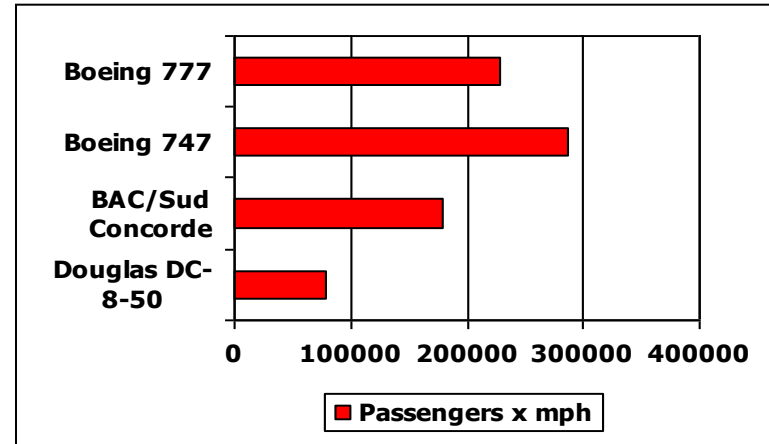
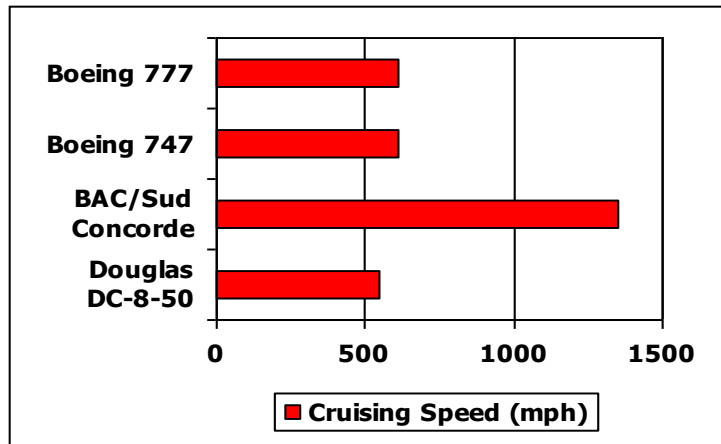
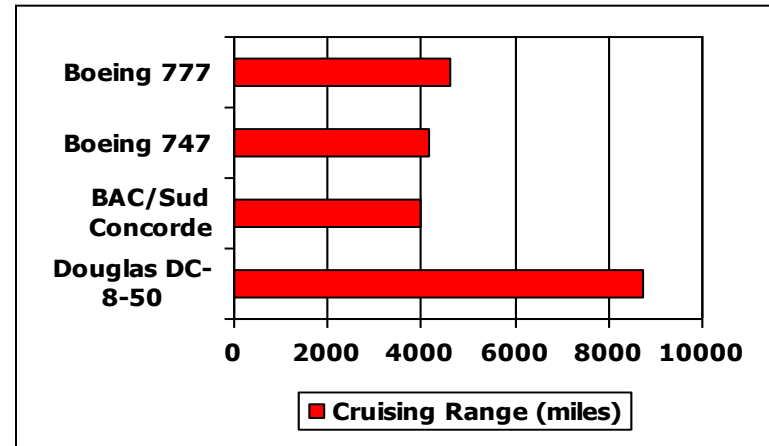
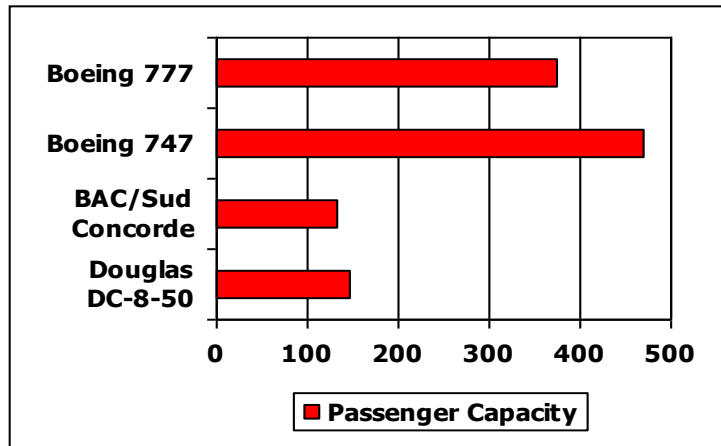
747-400:

- Capacity: 470 persons
- Range: 4150 miles
- Cruising speed: 567 mph (Mach 0.85) at 35,000 ft



Defining Performance

◆ Which airplane has the best performance?



差不多價錢的電腦，你怎麼比？



Response Time and Throughput

- ◆ **Response time** *CPU: 在意單核心能力*
 - How long it takes to do a task
- ◆ **Throughput** *GPU: 螞蟻雄兵; 2000-4000 cores*
 - Total work done per unit time
 - Ex: tasks/transactions/... per hour
- ◆ **How are response time and throughput affected by**
 - Replacing the processor with a faster version?
 - Adding more processors?
- ◆ **We'll focus on response time for now...**

Measuring Execution Time

◆ Elapsed time

- Total response time, including all aspects
 - Processing, I/O, OS overhead, idle time
- Determines system performance

◆ CPU time

- Time spent processing a given job
 - Discounts I/O time, other jobs' shares
- Comprises user CPU time and system CPU time
- Different programs are affected differently by CPU and system performance

Relative Performance

- ◆ Define Performance = $1/\text{Execution Time}$
- ◆ "X is n time faster than Y" (*Linux user mode : CPU Time*)

$$\begin{aligned} & \text{Performance}_X / \text{Performance}_Y \\ &= \text{Execution time}_Y / \text{Execution time}_X = n \end{aligned}$$

- ◆ Example: time taken to run a program
 - 10s on A, 15s on B
 - $\text{Execution Time}_B / \text{Execution Time}_A$
 $= 15s / 10s = 1.5$
 - So A is 1.5 times faster than B

什麼是電腦最基本的執行單位？

function

```
int fab(int x)
{
    if (x<=2) return 1;
    return fab(x-1) + fab(x-2);
}
```

```
lw    $15, 0($2)
lw    $16, 4($2)
sw    $16, 0($2)
sw    $15, 4($2)
```

instruction

loop

```
for(i=0; i<100; i++)
    sum += A[i];
```

```
temp = v[0];
v[0] = v[1];
v[1] = temp;
```

statement

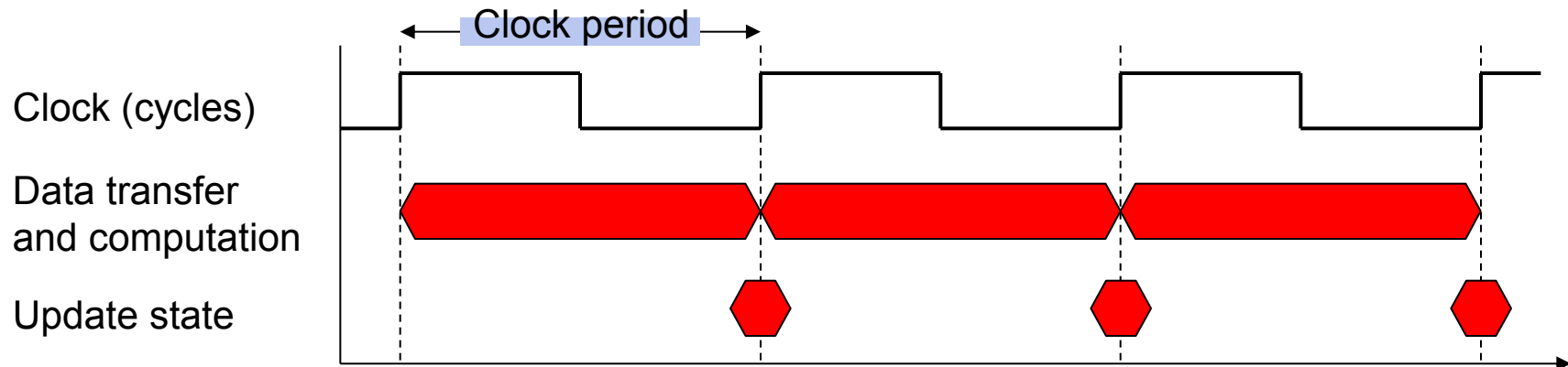


什麼是電腦最基本的時間單位？

答：「**clock cycle**」

CPU Clocking

- ◆ **Operation of digital hardware governed by a constant-rate clock**



- ◆ **Clock period: duration of a clock cycle**
 - **Ex: $250\text{ps} = 0.25\text{ns} = 250 \times 10^{-12}\text{s}$**
- ◆ **Clock frequency (rate): cycles per second**
 - **Ex: $4.0\text{GHz} = 4000\text{MHz} = 4.0 \times 10^9\text{Hz}$**

CPU Time

$$\begin{aligned} \text{CPU Time} &= \text{CPU Clock Cycles} \times \text{Clock Cycle Time} \\ &= \frac{\text{CPU Clock Cycles}}{\text{Clock Rate}} \end{aligned}$$

CC *T_c* *Freq*

- ◆ **Performance improved by**
 - Reducing number of clock cycles
 - Increasing clock rate (i.e. reducing clock cycle time)
 - Hardware designer must often trade off clock rate against cycle count

CPU Time Example

- ◆ **Computer A: 2GHz clock, 10s CPU time**
- ◆ **Designing Computer B**
 - **Aim for 6s CPU time**
 - **Can do faster clock, but causes $1.2 \times$ clock cycles**
- ◆ **How fast must Computer B clock be?**

$$\text{Clock Rate}_B = \frac{\text{Clock Cycles}_B}{\text{CPU Time}_B} = \frac{1.2 \times \text{Clock Cycles}_A}{6s}$$

$$\begin{aligned}\text{Clock Cycles}_A &= \text{CPU Time}_A \times \text{Clock Rate}_A \\ &= 10s \times 2\text{GHz} = 20 \times 10^9\end{aligned}$$

$$\text{Clock Rate}_B = \frac{1.2 \times 20 \times 10^9}{6s} = \frac{24 \times 10^9}{6s} = 4\text{GHz}$$

何謂程式執行的指令數？

- ◆ 有幾條C指令被執行？

```
a = b - c;  
for(i=a; i>0; i--)  
    sum = sum + x;
```

- ◆ 有幾條組合語言指令被執行？

```
                sub    $r1,$r2,$r3  
Loop:          blez   $r1,$r0,End  
                add    $r8,$r8,$r10  
                addi   $r1,$r1,-1  
                j      Loop
```

a times → $4 \times a + 2$ instructions

End:

Dynamic Instruction Count



指令需要幾個時間單位執行？

答：不一定（與指令複雜度和處理器設計有關）

Instruction Count and CPI

IC

CPI

Clock Cycles = Instruction Count $\times$ Cycles per Instruction

CPU Time = Instruction Count $\times$ CPI $\times$ Clock Cycle Time

$$= \frac{\text{Instruction Count} \times \text{CPI}}{\text{Clock Rate}}$$

- ◆ **Instruction Count for a program**
 - **Determined by program, ISA, and compiler**
- ◆ **Average cycles per instruction (CPI)**
 - **Determined by CPU hardware**
 - **If different instructions have different CPI**
 - **Average CPI affected by instruction mix**

(不同程式有不同平均CPI)

CPI Example

- ◆ **Computer A: Cycle Time = 250ps, CPI = 2.0**
- ◆ **Computer B: Cycle Time = 500ps, CPI = 1.2**
- ◆ **Same ISA**
- ◆ **Which one is faster, and by how much?**

$$\begin{aligned}\text{CPU Time}_A &= \text{Instruction Count} \times \text{CPI}_A \times \text{Cycle Time}_A \\ &= 1 \times 2.0 \times 250\text{ps} = 1 \times 500\text{ps} \leftarrow\end{aligned}$$

A is faster...

$$\begin{aligned}\text{CPU Time}_B &= \text{Instruction Count} \times \text{CPI}_B \times \text{Cycle Time}_B \\ &= 1 \times 1.2 \times 500\text{ps} = 1 \times 600\text{ps}\end{aligned}$$

$$\frac{\text{Performance}_A}{\text{Performance}_B} = \frac{\text{CPU Time}_B}{\text{CPU Time}_A} = \frac{1 \times 600\text{ps}}{1 \times 500\text{ps}} = 1.2 \leftarrow$$

...by this much

CPI in More Detail

- ◆ If different instruction classes take different numbers of cycles

$$\text{Clock Cycles} = \sum_{i=1}^n (\text{CPI}_i \times \text{Instruction Count}_i)$$

← 輕重! (weight)

- ◆ Weighted average CPI

← 加權 (weighted)

$$\overline{\text{CPI}} = \frac{\text{Clock Cycles}}{\text{Instruction Count}} = \sum_{i=1}^n \left(\text{CPI}_i \times \frac{\text{Instruction Count}_i}{\text{Instruction Count}} \right)$$

Relative frequency (ratio)

CPI Example

- ◆ Two compiled code sequences using instructions in classes A, B, C
- ◆ What are the average CPI of Sequence 1 and 2?

Class	A	B	C
CPI for class	1	2	3
IC in sequence 1	2	1	2
IC in sequence 2	4	1	1

- ◆ Sequence 1: IC = 5
 - Clock Cycles
 $= 2 \times 1 + 1 \times 2 + 2 \times 3$
 $= 10$
 - Avg. CPI = $10/5 = 2.0$

<

>

- ◆ Sequence 2: IC = 6
 - Clock Cycles
 $= 4 \times 1 + 1 \times 2 + 1 \times 3$
 $= 9$ ✓
 - Avg. CPI = $9/6 = 1.5$

Performance Summary

The BIG Picture

$$\text{CPU Time} = \frac{\overset{IC}{\text{Instructions}}}{\text{Program}} \times \frac{\overset{\overline{CPI}}{\text{Clock cycles}}}{\text{Instruction}} \times \frac{\overset{T_{\text{cycle}}}{\text{Seconds}}}{\text{Clock cycle}}$$

◆ Performance depends on

	Instruction Count	CPI	Clock Rate
Program	X	X	
Compiler	X	X	
Instruction Set	X	X	
Organization		X	X
Technology			X

ex. Intel 的 compiler 可提供 5-20% speedup.

CISC 大
RISC 小

← →
權衡

now fabless
Intel, AMD: IDM
(Integrated Design & Manufacture)

有關效能的另一個公式

從台北到高雄要多久？

4小時

0.5小時

0.5小時

如果改坐飛機，
台北到高雄只要1小時
全程可以加快多少？

應該如何推導公式？

由台北到高雄

◆ 不能enhance的部份為在市區的時間: $0.5 + 0.5 = 1$ 小時

◆ 可以enhance的部份為在高速公路上的4小時

=> 佔總時間的 $4/(4+1) = 0.8 = F$

◆ 現在改用飛機, 可以enhance的部份縮短為1小時

=> $S = 4/1 = 4$ (飛機快4倍)

◆ (整體) 走高速公路所需時間 $4 + 1$
 $\text{speedup} = \frac{\text{走高速公路所需時間}}{\text{坐飛機所需時間}} = \frac{4 + 1}{1 + 1} = 2.5$

◆ 另一種算法:

$\text{speedup} = \frac{1}{((1 - 0.8) + \frac{0.8}{4})} = \frac{1}{(1 - 0.8) + 0.2} = 2.5$

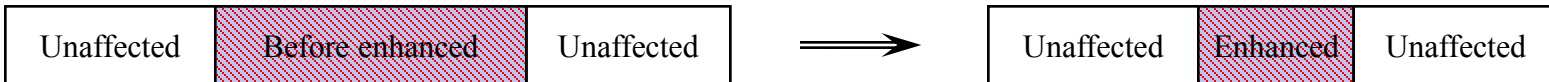
(50% 50%)

◆ When $S \rightarrow \infty$, $\text{speedup} \rightarrow 5$ 加速4倍
 應先改善市區交通 (持續去改善瓶頸)

Amdahl's Law

◆ Speedup due to enhancement E:

$$\text{Speedup}(E) = \frac{\text{Execution Time w/o } E}{\text{Execution Time w/ } E} = \frac{\text{Performance w/ } E}{\text{Performance w/o } E}$$



Suppose that enhancement E accelerates a fraction F of the task by a factor S and the remainder of the task is unaffected then,

$$\text{Execution Time(w/ } E) = \left((1 - F) + \frac{F}{S} \right) \times \text{Execution Time(w/o } E)$$

$$\text{Speedup(w/ } E) = \frac{1}{(1 - F) + \frac{F}{S}} \quad S \rightarrow \infty \approx \frac{1}{1 - F}$$

另一個與效能評比有關的議題...

- ◆ 使用什麼程式來評比才公平？
 - **Windows/Linux**開機？
 - **Windows/Linux**關機？
 - 九九乘法表？
 - 計組的期末程式？
 - 吃雞?!

What Programs for Comparison?

- ◆ **What's wrong with this program as a workload?**

```
integer A[][], B[][], C[][];  
for (I=0; I<100; I++)  
    for (J=0; J<100; J++)  
        for (K=0; K<100; K++)  
            C[I][J] = C[I][J] + A[I][K]*B[K][J];
```

- ◆ **What measured? Not measured? What is it good for?**

- ◆ **Ideally run typical programs with typical input before purchase, or before even build machine**

- **Called a "workload"; For example:**
- **Engineer uses compiler, spreadsheet**
- **Author uses word processor, drawing program, compression software**

Choosing Benchmark Programs

Pros

- representative
- portable
- widely used
- improvements useful in reality
- easy to run, early in design
- find potential bottleneck & peak capability

Actual Target Workload

Full Application Benchmarks

Small Kernel Benchmarks

Microbenchmarks

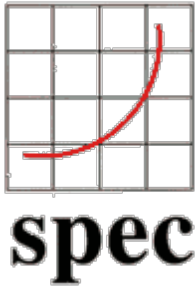
Cons

- not portable
- hard to measure, find cause
- less representative
- easy to fool
- peak does not reflect application performance

Benchmarks

- ◆ Obviously, apparent speed of processor depends on code used to test it
- ◆ Need industry standards so that different processors can be fairly compared => *benchmark programs*
- ◆ Companies exist that create these benchmarks: “typical” code used to evaluate systems
- ◆ Tricks in benchmarking:
 - different system configurations
 - compiler and libraries optimized (perhaps manually) for benchmarks
 - test specification biased towards one machine
 - very small benchmarks used
- ◆ Need to be changed every 2 or 3 years since designers could target these standard benchmarks

Example Standardized Workload Benchmarks



- ◆ **Standard Performance Evaluation Corporation (SPEC) : supported by a number of computer vendors to create standard set of benchmarks**
- ◆ **Began in 1989 focusing on benchmarking workstation and servers using CPU-intensive benchmarks**
- ◆ **The latest release: SPEC2022 benchmarks**
 - CPU
 - Cloud
 - Graphics and Workstation
 - High-performance computing / Parallel computing
 - Java client/server
 - Mail server
 - Storage
 - Power
 - Virtualization
 - Web server ...

URL: <https://www.spec.org/benchmarks.html>

Summarizing Performance

	Machine A	Machine B
Program 1	1 s	10 s
Program 2	1000 s	100 s
Total	1001 s	110 s

- ◆ A is 10 times faster than B for program 1
- ◆ B is 10 times faster than A for program 2
- What is relative performance of A & B?
- ◆ Arithmetic mean (tracking total time) assuming running programs 1 and 2 an equal number of times:

$$\frac{\text{Performance(B)}}{\text{Performance(A)}} = \frac{1001}{110} = 9.1$$

- ◆ Weighted arithmetic mean

SPEC CPU Benchmark

◆ SPEC CPU2006

- Elapsed time to execute a selection of programs
 - Negligible I/O, so focuses on CPU performance
- Normalize relative to reference machine
- Summarize as geometric mean of performance ratios
 - CINT2006 (integer) and CFP2006 (floating-point)

$$\sqrt[n]{\prod_{i=1}^n \text{Execution time ratio}_i}$$

乘起來 (幾何平均)

CINT2006 for Opteron X4 2356

Name	Description	IC $\times 10^9$	CPI	Tc (ns)	Exec time	Ref time	SPECratio
perl	Interpreted string processing	2,118	0.75	0.40	637	9,777	15.3
bzip2	Block-sorting compression	2,389	0.85	0.40	817	9,650	11.8
gcc	GNU C Compiler	1,050	1.72	0.47	24	8,050	11.1
mcf	Combinatorial optimization	336	10.00	0.40	1,345	9,120	6.8
go	Go game (AI)	1,658	1.09	0.40	721	10,490	14.6
hmmer	Search gene sequence	2,783	0.80	0.40	890	9,330	10.5
sjeng	Chess game (AI)	2,176	0.96	0.48	37	12,100	14.5
libquantum	Quantum computer simulation	1,623	1.61	0.40	1,047	20,720	19.8
h264avc	Video compression	3,102	0.80	0.40	993	22,130	22.3
omnetpp	Discrete event simulation	587	2.94	0.40	690	6,250	9.1
astar	Games/path finding	1,082	1.79	0.40	773	7,020	9.1
xalancbmk	XML parsing	1,058	2.70	0.40	1,143	6,900	6.0
Geometric mean							11.7

High cache miss rates

SPEC Power 2008 Benchmark

- ◆ **Power consumption of server at different workload levels**
 - **Performance: ssj_ops/sec**
 - **Power: Watts (Joules/sec)**
 - **ssj stands for server-side Java**

$$\text{Overall ssj_ops per Watt} = \left(\sum_{i=0}^{10} \text{ssj_ops}_i \right) / \left(\sum_{i=0}^{10} \text{power}_i \right)$$

SPECpower_ssj2008 for X4

Target Load %	Performance (ssj_ops/sec)	Average Power (Watts)
100%	231,867	295
90%	211,282	286
80%	185,803	275
70%	163,427	265
60%	140,160	256
50%	118,324	246
40%	92,035	233
30%	70,500	222
20%	47,126	206
10%	23,066	180
0%	0	141
Overall sum	1,283,590	2,605
Σ ssj_ops/ Σ power		493

clock still
running

Outline

- ◆ **Computer: A historical perspective**

- ◆ **Moore's Law**

- ◆ **Abstractions**

- ◆ **Technology**

- **Performance**

- **Definition**

- **CPU performance**

- **Amdahl's Law**

- **Benchmarking**

- **Power wall**

- **A trend from uniprocessors to multiprocessors**

- **Manufacturing cost**

① *Power Wall*

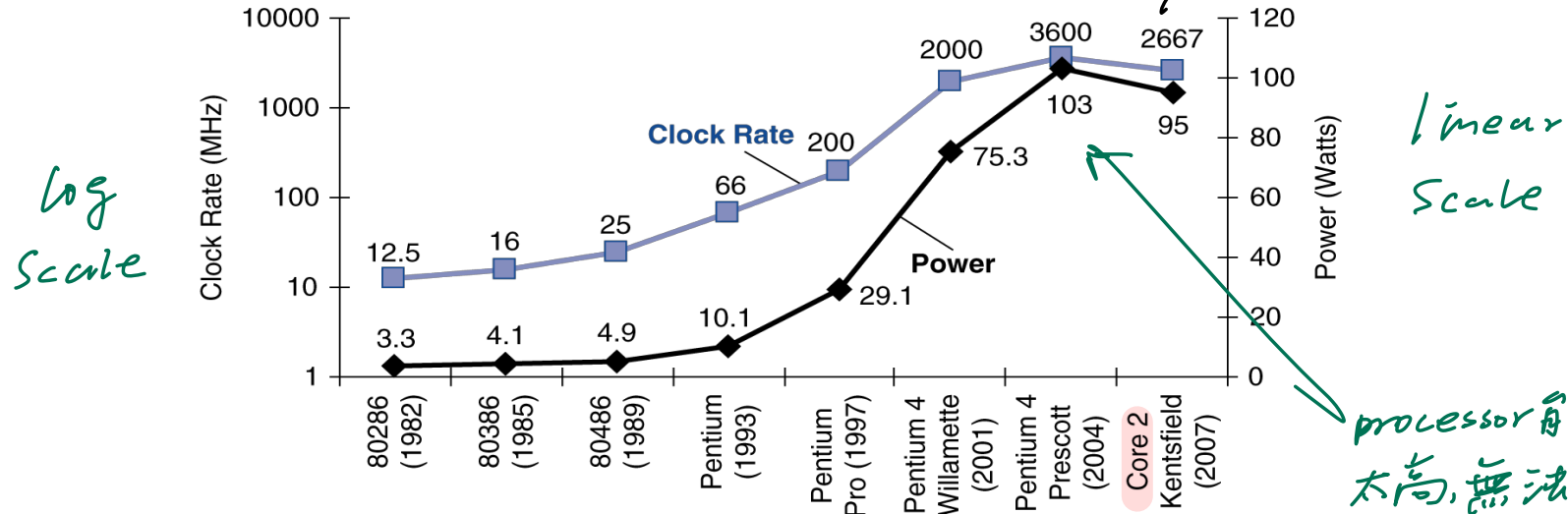
② *ILP Wall*

(instruction-level parallelism)

③ *Memory Wall*

Power Trends

Intel 時脈戰爭結束
⇒ 進入多核心時代



(功率)
processor 能量密度
太高, 無法再提升
時脈! (散熱...)

◆ In CMOS IC technology

R-C 电路:
电容充电: $\frac{1}{2}CV^2$ 存於电容, $\frac{1}{2}CV^2$ 耗於电阻。
电容放电: 放出 $\frac{1}{2}CV^2$, 耗於电阻

$$\text{Power} = \text{Capacitive load} \times \text{Voltage}^2 \times \text{Frequency}$$

(消耗) *energy*
time

~ 30x

电晶体变小
⇒ 更容易导电。

5V → 1V

电晶体 (电容)
充线次/时间

~ 1000x

Reducing Power

◆ Suppose a new CPU has

- 85% of capacitive load of old CPU
- 15% voltage and 15% frequency reduction

① 晶体管变小 $\Rightarrow$ 电容 $\downarrow$
 ② $\frac{\text{Gate}}{\text{SiO}_2}$ 绝缘层变薄 $\Rightarrow$ 电容 $\uparrow$

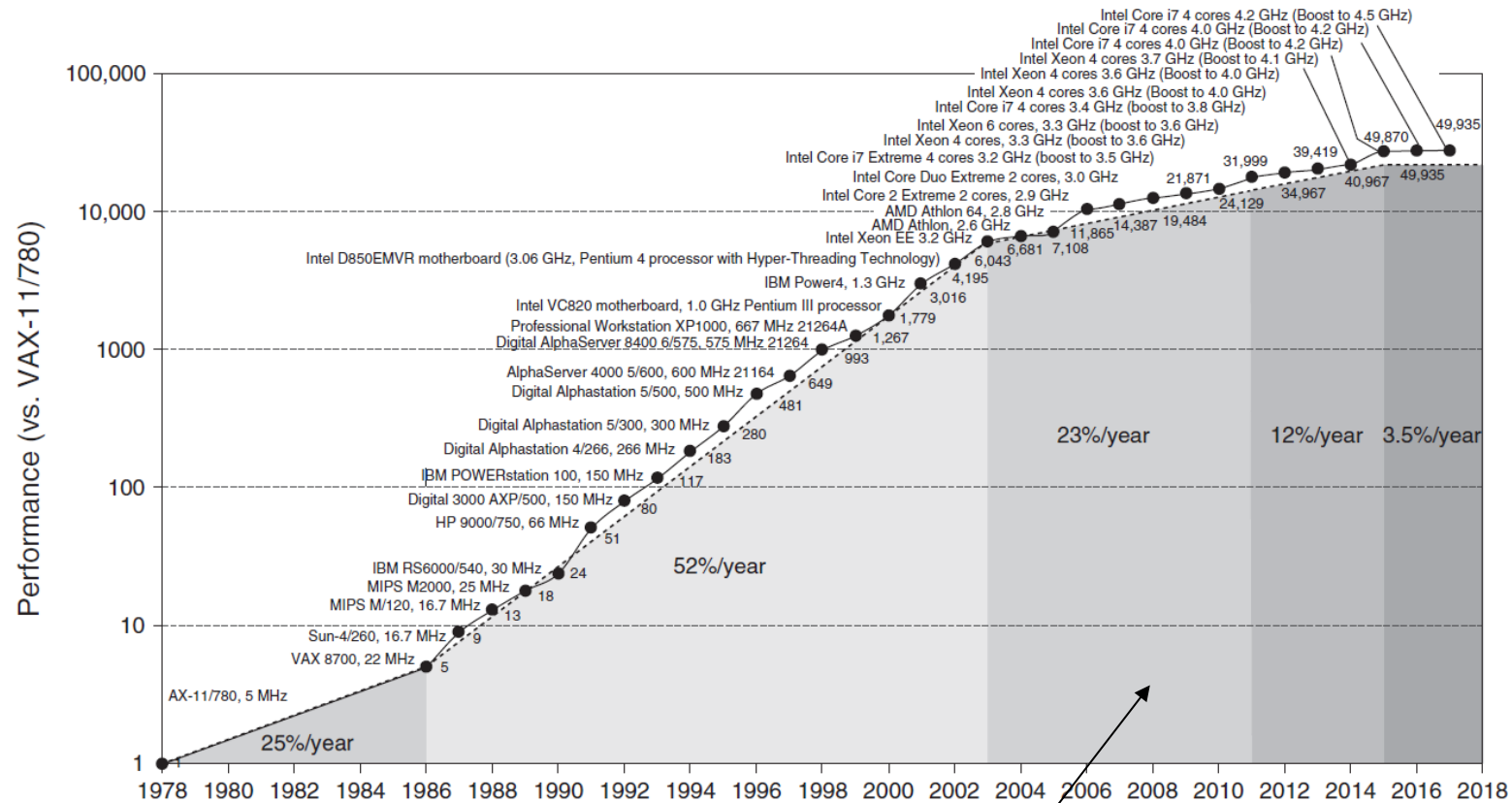
$$\frac{P_{\text{new}}}{P_{\text{old}}} = \frac{C_{\text{old}} \times 0.85 \times (V_{\text{old}} \times 0.85)^2 \times F_{\text{old}} \times 0.85}{C_{\text{old}} \times V_{\text{old}}^2 \times F_{\text{old}}} = 0.85^4 = 0.52$$

◆ The power wall

- We can't reduce voltage further
- We can't remove more heat

◆ How else can we improve performance?

Uniprocessor Performance



Constrained by power, instruction-level parallelism, memory latency

Multiprocessors

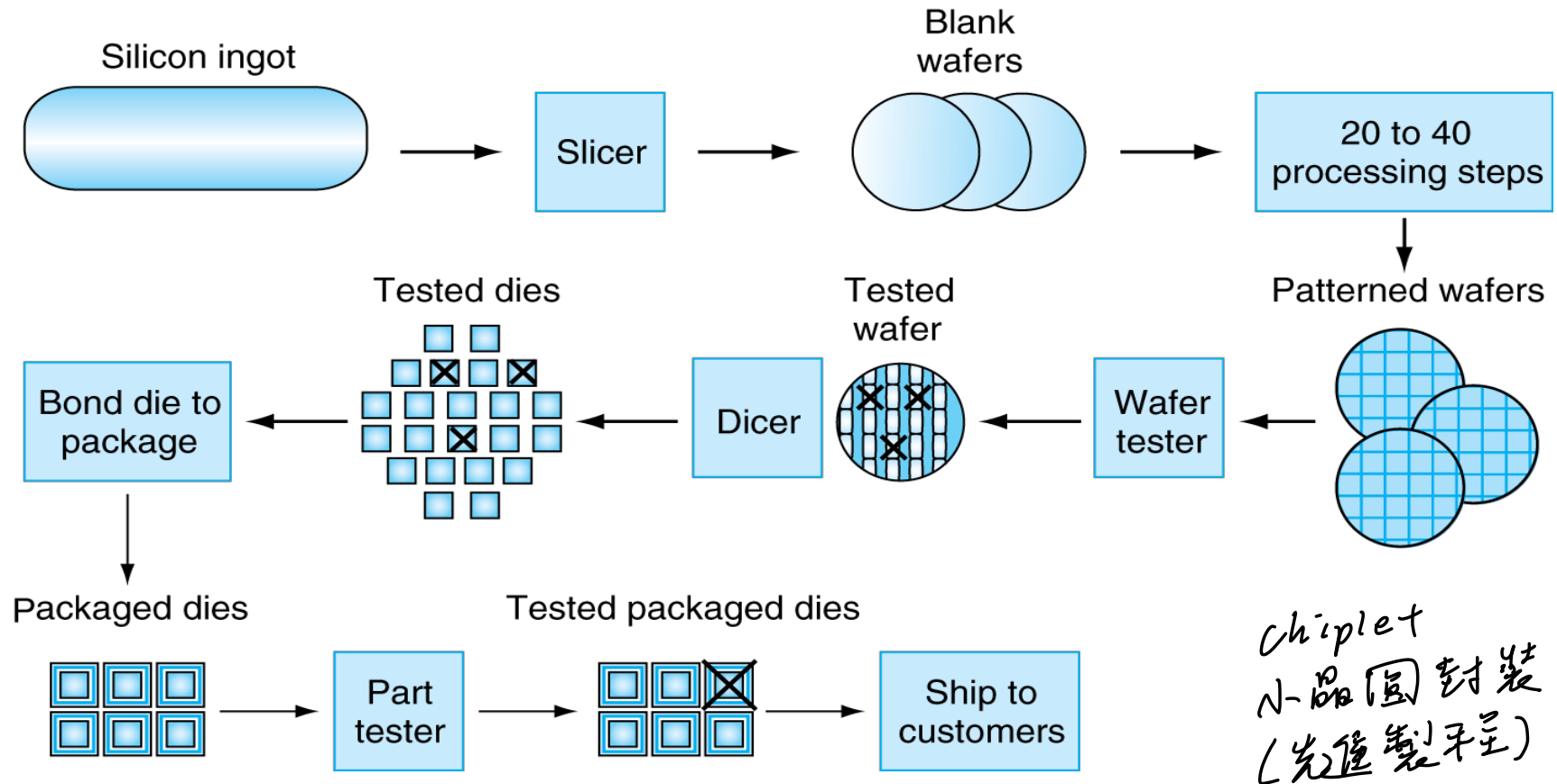
- ◆ **Multicore microprocessors**
 - **More than one processor per chip**
- ◆ **Requires explicitly parallel programming**
 - **Compare with instruction level parallelism**
 - **Hardware executes multiple instructions at once**
 - **Hidden from the programmer**
 - **Hard to do**
 - **Programming for performance**
 - **Load balancing**
 - **Optimizing communication and synchronization**

Outline



- ◆ **Computer: A historical perspective**
- ◆ **Moore's Law**
- ◆ **Abstractions**
- ◆ **Technology**
 - **Performance**
 - **Definition**
 - **CPU performance**
 - **Amdahl's Law**
 - **Benchmarking**
 - **Power wall**
 - **A trend from uniprocessors to multiprocessors**
 - **Manufacturing cost**

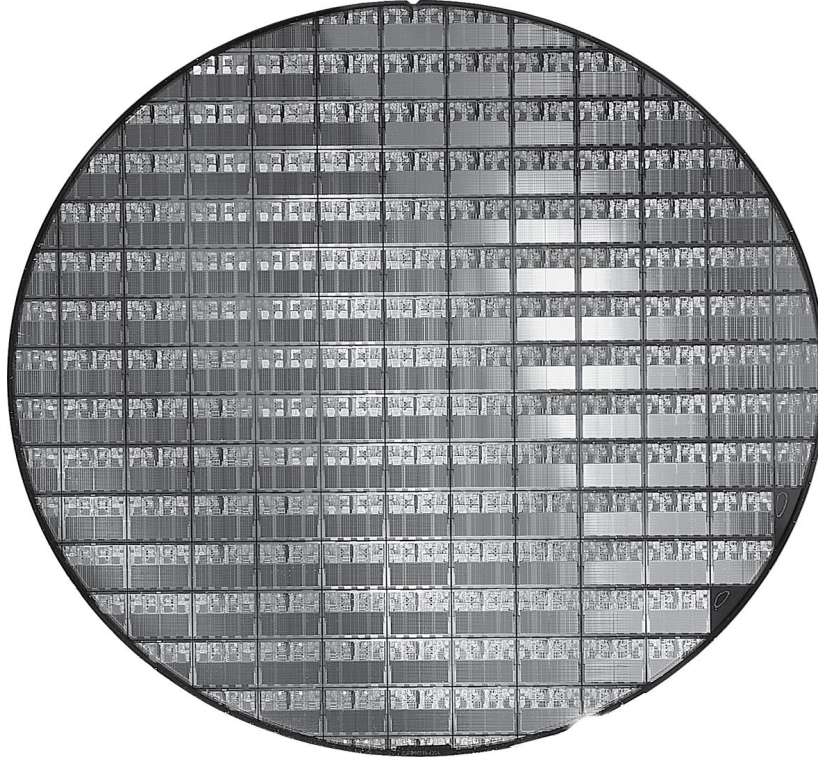
Manufacturing ICs



chiplet
 N-晶圆封装
 (先进制程)
 → CoWoS, (2D)
 InFO,
 SoIC (3D)

◆ **Yield: proportion of working dies per wafer**

AMD Opteron X2 Wafer



- ◆ **X2: 300mm wafer, 117 chips, 90nm technology**
- ◆ **X4: 45nm technology**

Integrated Circuit Cost

$$\text{Cost per die} = \frac{\text{Cost per wafer}}{\text{Dies per wafer} \times \text{Yield}}$$

$$\text{Dies per wafer} \approx \text{Wafer area} / \text{Die area}$$

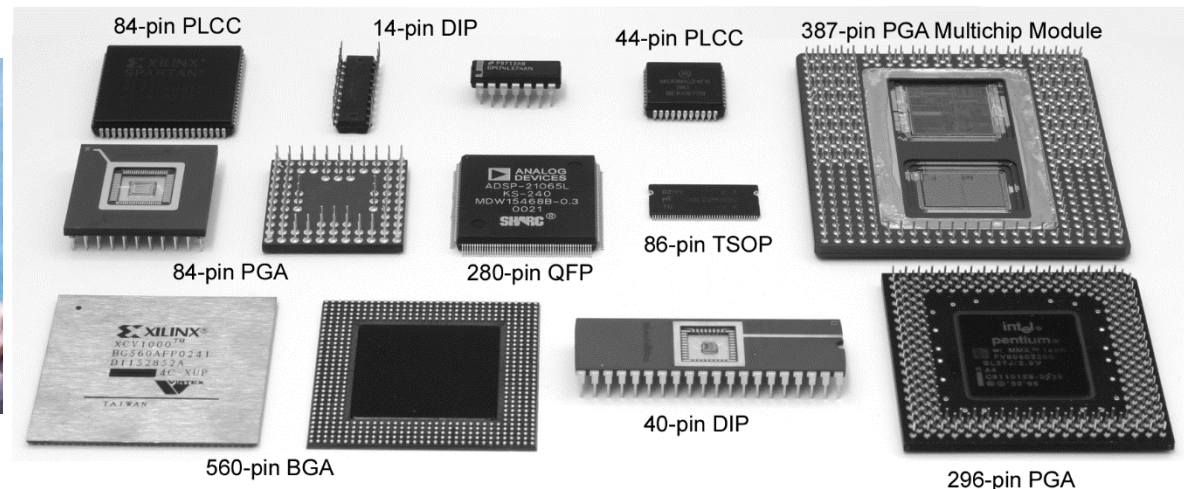
$$\text{Yield} = \frac{\# \text{ of good dies}}{\# \text{ of total dies}} = \frac{1}{(1 + (\text{Defects per area} \times \text{Die area}/2))^2}$$

- ◆ **Nonlinear relation to area and defect rate**
 - **Wafer cost and area are fixed**
 - **Defect rate determined by manufacturing process**
 - **Die area determined by architecture and circuit design**

Cost of a Chip Includes ...

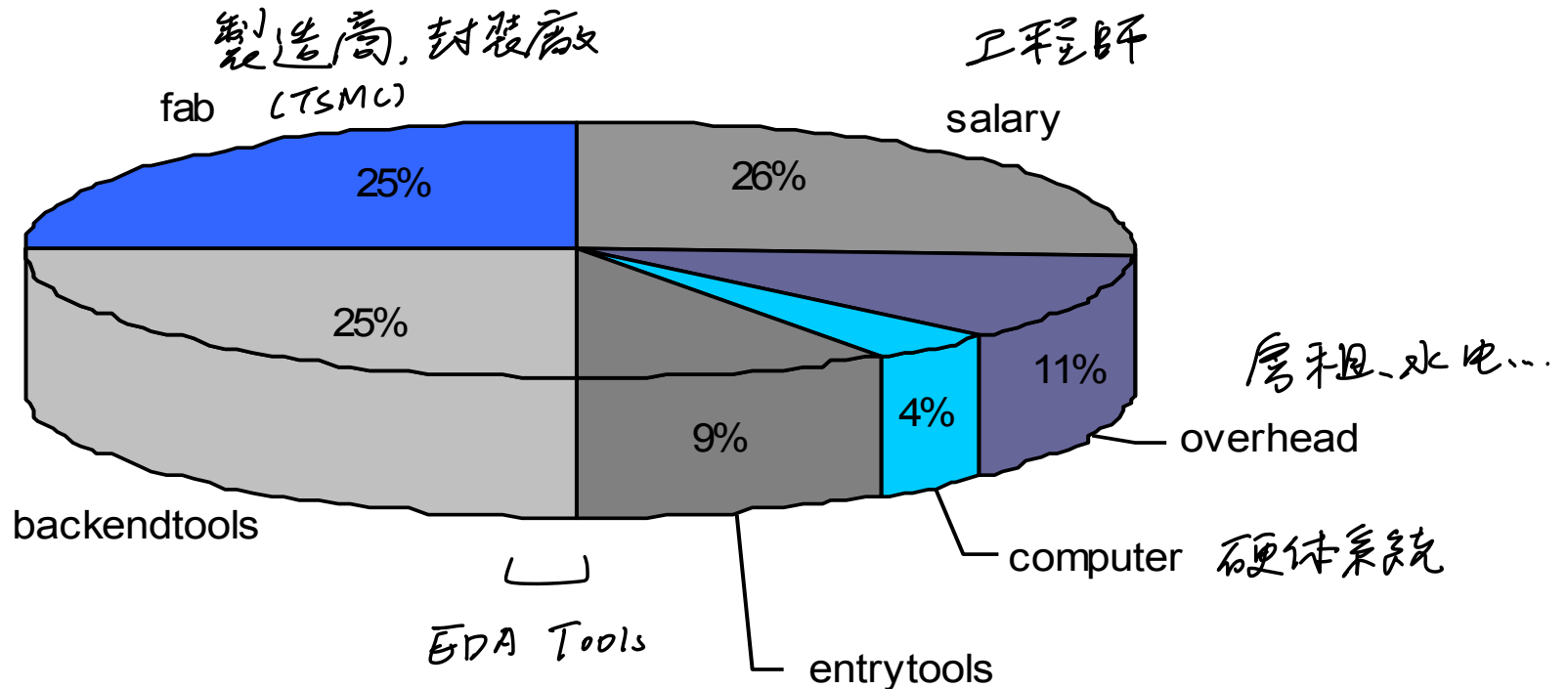
- ◆ **Die cost:** affected by wafer cost, number of dies per wafer, and die yield ($\frac{\text{\#good dies}}{\text{\#total dies}}$)
 - goes roughly with the cube of the die area
 - An 8" wafer can contain 196 Pentium dies, but only 78 Pentium Pro
- ◆ **Testing cost**
- ◆ **Packaging cost:** depends on pins, heat dissipation,

■ ■ ■



Cost Breakdown

- ◆ New chip design is fairly capital-intensive
- ◆ Maybe you can do it for less?



Pitfall: Amdahl's Law

- ◆ Improving an aspect of a computer and expecting a proportional improvement in overall performance

$$T_{\text{improved}} = \frac{T_{\text{affected}}}{\text{improvement factor}} + T_{\text{unaffected}}$$

- ◆ Example: multiply accounts for 80s/100s
 - How much improvement in multiply performance to get 5× overall?

$$20 = \frac{80}{n} + 20 \quad \bullet \text{ Cannot be done!}$$

- ◆ Corollary: make the common case fast

Fallacy: Low Power at Idle

- ◆ **Look back at X4 power benchmark**
 - **At 100% load: 295W**
 - **At 50% load: 246W (83%)**
 - **At 10% load: 180W (61%)**
- ◆ **Google data center**
 - **Mostly operates at 10% – 50% load**
 - **At 100% load less than 1% of the time**
- ◆ **Consider designing processors to make power proportional to load**

Pitfall: MIPS as a Performance Metric

(aka MIPS the ISA)

◆ **MIPS: Millions of Instructions Per Second**

- **Doesn't account for**
 - Differences in ISAs between computers
 - Differences in complexity between instructions

$$\begin{aligned} \text{MIPS} &= \frac{\text{Instruction count}}{\text{Execution time} \times 10^6} \\ &= \frac{\text{Instruction count}}{\frac{\text{Instruction count} \times \text{CPI}}{\text{Clock rate}} \times 10^6} = \frac{\text{Clock rate}}{\text{CPI} \times 10^6} \end{aligned}$$

IC 除掉了, 不能当效能指标
performance → CPU Time

- **CPI varies between programs on a given CPU**

Concluding Remarks

- ◆ **Cost/performance is improving**
 - Due to underlying technology development
- ◆ **Hierarchical layers of abstraction**
 - In both hardware and software
- ◆ **Instruction set architecture**
 - The hardware/software interface
- ◆ **Execution time: the best performance measure**
- ◆ **Power is a limiting factor**
 - Use parallelism to improve performance